



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SAVEETHA
SCHOOL OF ENGINEERING

COURSE CODE: ECA03

COURSE NAME: SIGNALS AND SYSTEMS

MASTER RECORD

S.No.	Title Of The Experiment	Page No
1	Generation of Continuous-Time Signals and Discrete-Time Signals	3
2	Periodic and Aperiodic Signals	7
3	Addition and Multiplication of Two Sequences	10
4	Causal / Non-Causal System	13
5	Autocorrelation and Cross-Correlation	16
6	Fourier Series of Half-Wave Rectifier Output	21
7	Fourier Transform of a Rectangular Waveform	24
8	Fourier Transform of Periodic Impulse Train	27
9	Analysis of Discrete Time System	30
10	Analysis of Continuous-Time System	33
11	Impulse Response of the System	36
12	Discrete-Time Fourier Transform (DTFT) and Discrete-Time Fourier Transform of Rectangular Pulse	39
13	Pole-Zero Plot	44
14	Z-Transform of the Given Signal	47
15	Convolution of Continuous-Time Signals and Discrete-Time Signals	51
16	Discrete Fourier Transform (DFT)	57
17	Circular Convolution	60
18	Step Response of the System	63
19	Generation of Triangular and Sawtooth Signals	66
20	Verification of Even and Odd Signal Decomposition	68
21	Energy and Power Calculation of Given Signals	70
22	Amplitude and Time Scaling of Signals	71
23	Time Shifting and Time Reversal of Signals	73
24	Sampling Theorem Verification and Aliasing Effect Demonstration	74
25	Reconstruction of Signal from Samples (Ideal Interpolation)	78
26	Step and Ramp Functions (Continuous & Discrete)	80
27	FFT – Decimation-In-Time (DIT) Algorithm	82
28	FFT – Decimation-In-Frequency (DIF) Algorithm	84
29	Fourier Transform of a Continuous-Time Signal	86
30	Verification of Linearity Property of LTI Systems	88

1.A. GENERATION OF CONTINUOUS TIME SIGNALS

Aim:

To verify the generation of a sinusoidal signal with a specific frequency, amplitude, and phase shift using Scilab. The Aim is to ensure that the signal generator can accurately produce a sinusoidal waveform with the desired parameters.

Test Parameters:

- **Frequency (f):** 5 Hz
- **Amplitude (A):** 2
- **Phase Shift (ϕ):** $\pi/4$ radians
- **Time Duration:** 0 to 1 second

Program:

```
// Test Case: Sinusoidal Signal Generation

// Define parameters
f = 5;           // Frequency in Hz
A = 2;           // Amplitude
phi = %pi/4;     // Phase shift in radians
t = 0:0.001:1;   // Time vector from 0 to 1 second with step size 0.001

// Generate the sinusoidal signal
x_t = A * sin(2 * %pi * f * t + phi);

// Plot the signal
clf; // Clear the current figure
plot(t, x_t);
xlabel('Time (s)');
ylabel('Amplitude');
title('Sinusoidal Signal: 5 Hz, Amplitude 2, Phase Shift  $\pi/4$ ');
xtitle('Continuous Time Sinusoidal Signal','Time (s)','Amplitude');
```

Expected Output:

When you run the Scilab code, you should observe a sinusoidal waveform plotted against time. The signal will have the following characteristics:

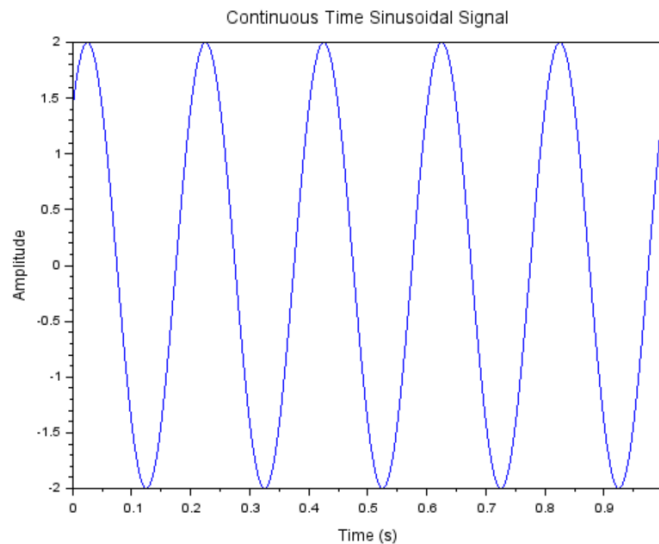
- **Frequency:** The signal completes 5 full cycles within 1 second (5 Hz).
- **Amplitude:** The peak value of the waveform should be 2 (positive peak) and -2 (negative peak).

Here's a brief description of the expected plot:

- The x-axis represents time (from 0 to 1 second).

- The y-axis represents the amplitude of the signal (ranging from -2 to 2).
- The waveform should start at a positive value and complete 5 full oscillations within the time span of 1 second.

Output Waveform:



Result:

This test case validates that the signal generator can produce a sinusoidal waveform with the specified frequency, amplitude, and phase shift.

1.B. GENERATION OF DISCRETE TIME SIGNALS

Discrete-Time Unit Impulse Signal Generation

Aim:

To generate and verify a discrete-time unit impulse signal using Scilab. The Aim is to ensure that the signal generator can accurately produce an impulse signal at a specified time index, which is crucial for analyzing and understanding the behavior of discrete-time systems.

Test Parameters:

- **Length of Signal (N):** 21 samples
- **Impulse Location (n0):** 11 (to place the impulse at the center of the signal)

Program:

```
// Test Case: Discrete-Time Unit Impulse Signal Generation
```

```
// Define parameters
```

```
N = 21;           // Length of the signal
```

```
n0 = 11;          // Index at which the impulse occurs
```

```

n = 1:N;          // Discrete time index vector

// Generate the unit impulse signal
x_n = zeros(1, N); // Initialize signal with zeros
x_n(n0) = 1;       // Set the impulse at the desired location

// Plot the signal
clf; // Clear the current figure
plot2d(n, x_n);
xlabel('n (Discrete Time Index)');
ylabel('Amplitude');
title('Discrete-Time Unit Impulse Signal');
xtitle('Discrete-Time Unit Impulse Signal','n (Discrete Time Index)','Amplitude');

```

Expected Output:

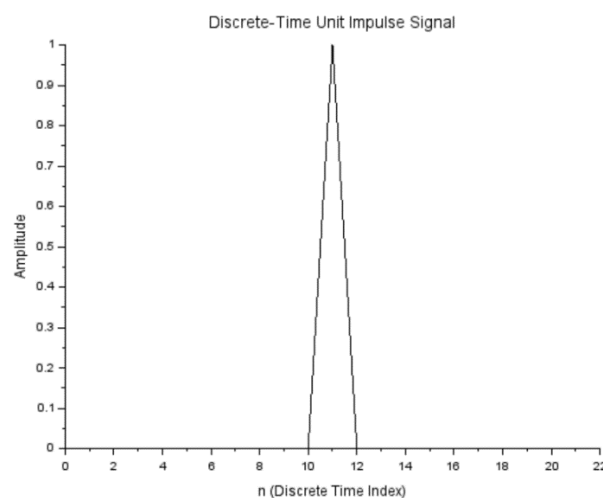
When you run the Scilab code, you should observe a stem plot of the unit impulse signal. The signal will have the following characteristics:

- **Length:** The signal consists of 21 samples.
- **Impulse Location:** The impulse will be at the 11th position (center of the signal), with a value of 1.
- **Amplitude:** All other samples will have an amplitude of 0, except for the impulse at index 11, which will have an amplitude of 1.

Here's a brief description of the expected plot:

- The x-axis represents the discrete time index n (ranging from 1 to 21).
- The y-axis represents the amplitude of the signal (with values 0 or 1).
- The plot will show a single vertical line (stem) at $n=11$ with an amplitude of 1, and all other values will be at 0.

Output Waveform:



Result:

This test case validates that the signal generator can accurately produce a discrete-time unit impulse signal at a specified index, which is essential for analyzing the response of discrete-time systems.

2. PERIODIC AND APERIODIC SIGNALS

Generation and Analysis of Periodic Signal (Sinusoidal Wave) and Aperiodic Signals (Exponentially Decaying Signal)

Aim:

To generate and distinguish between periodic and aperiodic signals using Scilab. The Aim is to demonstrate the fundamental difference between periodic and aperiodic signals by creating examples of each and analyzing their properties.

Test Parameters:

1. **Periodic Signal:**
 - **Type:** Sinusoidal Signal
 - **Frequency (f):** 1 Hz
 - **Amplitude (A):** 1
 - **Time Duration:** 0 to 2 seconds
2. **Aperiodic Signal:**
 - **Type:** Exponentially Decaying Signal
 - **Base of Exponential (a):** 0.5
 - **Amplitude (A):** 2
 - **Time Duration:** 0 to 2 seconds

Program:

```
// Test Case: Generation of Periodic and Aperiodic Signals

// Parameters for the periodic signal (Sinusoidal)
f_periodic = 1;    // Frequency in Hz
A_periodic = 1;    // Amplitude
t_periodic = 0:0.01:2; // Time vector for 0 to 2 seconds

// Generate the periodic signal
x_periodic = A_periodic * sin(2 * %pi * f_periodic * t_periodic);

// Plot the periodic signal
clf; // Clear the current figure
subplot(2,1,1);    // Subplot 1 for the periodic signal
plot(t_periodic, x_periodic);
xlabel('Time (s)');
ylabel('Amplitude');
title('Periodic Signal: Sinusoidal Wave (1 Hz)');

// Parameters for the aperiodic signal (Exponentially Decaying)
a_aperiodic = 0.5; // Base of the exponential decay
A_aperiodic = 2;    // Amplitude
t_aperiodic = 0:0.01:2; // Time vector for 0 to 2 seconds
```

```
// Generate the aperiodic signal
x_aperiodic = A_aperiodic * exp(-a_aperiodic * t_aperiodic);

// Plot the aperiodic signal
subplot(2,1,2);    // Subplot 2 for the aperiodic signal
plot(t_aperiodic, x_aperiodic);
xlabel('Time (s)');
ylabel('Amplitude');
title('Aperiodic Signal: Exponentially Decaying Signal');
```

Expected Output:

When you run the Scilab code, you should observe two plots:

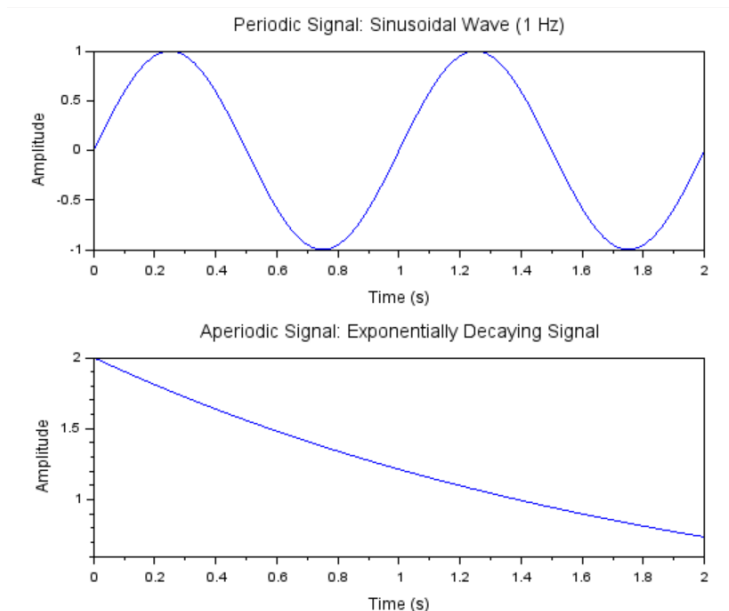
1. Periodic Signal (Sinusoidal Wave):

- **Characteristics:** The first subplot shows a sinusoidal waveform that repeats its pattern every 1 second, as it is a 1 Hz sinusoid.
- **Description:** The signal oscillates between -1 and 1, completing two full cycles over the 2-second duration.

2. Aperiodic Signal (Exponentially Decaying Signal):

- **Characteristics:** The second subplot shows an exponentially decaying signal. Unlike the sinusoidal signal, this signal does not repeat its pattern.
- **Description:** The signal starts at amplitude 2 and decays towards zero as time progresses, reflecting its aperiodic nature.

Output Waveform:



Result:

- **Periodic Signal:** The sinusoidal signal repeats itself at regular intervals, making it periodic.
- **Aperiodic Signal:** The exponentially decaying signal does not repeat, showcasing that it is aperiodic.

This test case illustrates the key differences between periodic and aperiodic signals by generating and visually analyzing both types using Scilab.

3. ADDITION AND MULTIPLICATION OF TWO SEQUENCES

Addition and Multiplication of Two Sequences (Sinusoidal Sequence and Exponentially Decaying Sequence)

Aim:

To perform and verify the addition and multiplication of two discrete-time sequences using Scilab. The Aim is to understand how basic arithmetic operations on sequences work and to observe the resulting sequences after performing addition and multiplication.

Test Parameters:

- **Sequence 1 ($x_1[n]$):** A sinusoidal sequence
- **Sequence 2 ($x_2[n]$):** An exponentially decaying sequence
- **Number of Samples (N):** 30 samples

Program:

```
// Test Case: Addition and Multiplication of Two Sequences
```

```
// Define parameters
```

```
N = 30;           // Number of samples
```

```
n = 0:N-1;       // Discrete time index vector
```

```
// Define Sequence 1: Sinusoidal sequence
```

```
f1 = 0.05;        // Frequency in cycles/sample
```

```
A1 = 2;           // Amplitude
```

```
x1_n = A1 * sin(2 * %pi * f1 * n);
```

```
// Define Sequence 2: Exponentially decaying sequence
```

```
a2 = 0.1;         // Base of the exponential decay
```

```
A2 = 3;           // Amplitude
```

```
x2_n = A2 * exp(-a2 * n);
```

```
// Addition of the two sequences
```

```
x_add = x1_n + x2_n;
```

```
// Multiplication of the two sequences
```

```
x_mult = x1_n .* x2_n;
```

```
// Plot the sequences and their results
```

```
// Plot Sequence 1
```

```
clf; // Clear the current figure
```

```
subplot(4,1,1);
```

```

stem(n, x1_n);
xlabel('n (Discrete Time Index)');
ylabel('Amplitude');
title('Sequence 1: Sinusoidal Sequence');

// Plot Sequence 2
subplot(4,1,2);
stem(n, x2_n);
xlabel('n (Discrete Time Index)');
ylabel('Amplitude');
title('Sequence 2: Exponentially Decaying Sequence');

// Plot the result of addition
subplot(4,1,3);
stem(n, x_add);
xlabel('n (Discrete Time Index)');
ylabel('Amplitude');
title('Addition of Sequence 1 and Sequence 2');

// Plot the result of multiplication
subplot(4,1,4);
stem(n, x_mult);
xlabel('n (Discrete Time Index)');
ylabel('Amplitude');
title('Multiplication of Sequence 1 and Sequence 2');

```

Expected Output:

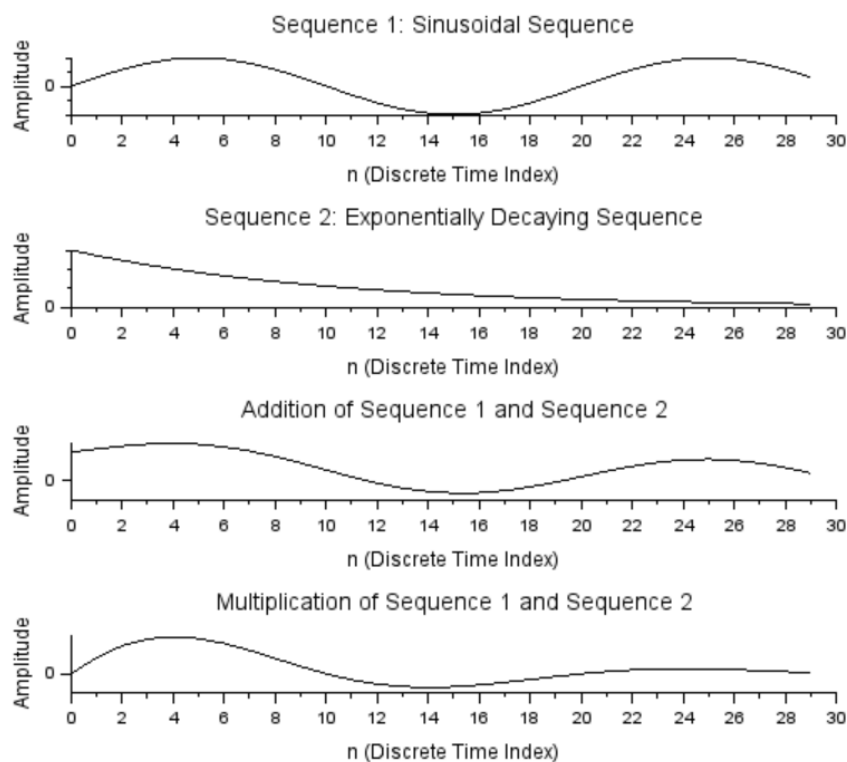
When you run the Scilab code, you should observe four subplots:

1. **Sequence 1 (Sinusoidal Sequence):**
 - **Characteristics:** The first subplot shows a sinusoidal sequence with 30 samples. The sequence oscillates between positive and negative values, reflecting a sinusoidal waveform.
 - **Description:** The sequence is periodic with a frequency of 0.05 cycles/sample.
2. **Sequence 2 (Exponentially Decaying Sequence):**
 - **Characteristics:** The second subplot shows an exponentially decaying sequence. The sequence starts at a higher value and decays towards zero as the sample index increases.
 - **Description:** The sequence is aperiodic and decreases in amplitude with time.
3. **Addition of Sequences:**
 - **Characteristics:** The third subplot shows the result of adding Sequence 1 and Sequence 2. The resultant sequence combines the oscillatory nature of the sinusoid with the decaying trend of the exponential sequence.
 - **Description:** The sequence's overall amplitude is influenced by both sequences, resulting in a waveform that decays over time but retains the sinusoidal shape.

4. Multiplication of Sequences:

- **Characteristics:** The fourth subplot shows the result of multiplying Sequence 1 and Sequence 2. The resulting sequence has a waveform that is both oscillatory and exponentially decaying.
- **Description:** The sequence's amplitude decreases over time due to the multiplication by the exponentially decaying sequence, while the sinusoidal pattern is maintained

Output Waveform:



Result:

- **Addition:** The addition of the two sequences combines their characteristics, leading to a sequence that reflects the contributions of both the sinusoidal and exponential components.
- **Multiplication:** The multiplication of the two sequences results in a sequence that has the characteristics of both, where the sinusoidal sequence is modulated by the exponential decay.

This test case demonstrates the process of adding and multiplying two sequences in the discrete-time domain, providing insight into how these operations affect the resulting signals.

4. CAUSAL/NON-CAUSAL SYSTEM

Analysis of Causal $y[n] = x[n] + 0.5x[n-1]$ and Non-Causal Systems $y[n] = 0.5x[n] + 0.5x[n+1]$

Aim:

To analyze and distinguish between causal and non-causal discrete-time systems. The Aim is to understand how causality affects the system's response to different input sequences.

System Definitions:

1. Causal System:

- Difference Equation: $y[n] = x[n] + 0.5x[n-1]$
- Input Sequence ($x[n]$): A unit step sequence

2. Non-Causal System:

- Difference Equation: $y[n] = 0.5x[n] + 0.5x[n+1]$
- Input Sequence ($x[n]$): A unit step sequence
- Number of Samples (N): 20 samples

Program:

// Test Case: Causal and Non-Causal Systems Analysis

// Define parameters

```
N = 20;           // Number of samples
n = 0:N-1;        // Discrete time index vector
x_n = ones(1, N); // Input sequence: Unit step sequence
```

// Initialize output sequences

```
y_causal = zeros(1, N); // Output sequence for causal system
y_noncausal = zeros(1, N); // Output sequence for non-causal system
```

// Causal System: $y[n] = x[n] + 0.5x[n-1]$

```
for k = 1:N
    if k > 1 then
        y_causal(k) = x_n(k) + 0.5 * x_n(k-1);
    else
        y_causal(k) = x_n(k); // No x[n-1] for k=0
    end
end
```

```

// Non-Causal System:  $y[n] = 0.5x[n] + 0.5x[n+1]$ 
for k = 1:N
    if k < N then
        y_noncausal(k) = 0.5 * x_n(k) + 0.5 * x_n(k+1);
    else
        y_noncausal(k) = 0.5 * x_n(k); // No  $x[n+1]$  for  $k=N-1$ 
    end
end

// Plot the input sequence
clf; // Clear the current figure
subplot(3,1,1);
plot2d(n, x_n);
xlabel('n (Discrete Time Index)');
ylabel('Amplitude');
title('Input Sequence: Unit Step Sequence');

// Plot the output of the causal system
subplot(3,1,2);
plot2d(n, y_causal);
xlabel('n (Discrete Time Index)');
ylabel('Amplitude');
title('Output Sequence: Causal System');

// Plot the output of the non-causal system
subplot(3,1,3);
plot2d(n, y_noncausal);
xlabel('n (Discrete Time Index)');
ylabel('Amplitude');
title('Output Sequence: Non-Causal System');

```

Expected Output:

When you run the Scilab code, you should observe three subplots:

1. Input Sequence (Unit Step Sequence):

- **Characteristics:** The first subplot shows a unit step sequence where the amplitude is 1 for all $n \geq 0$ and 0 for $n < 0$. Since the sequence starts at $n=0$, all values are 1.
- **Description:** This sequence is used as the input to both systems.

2. Output of the Causal System:

- **Characteristics:** The second subplot shows the output of the causal system, defined by $y[n] = x[n] + 0.5x[n-1]$. The output at each time index depends only on the current and previous input values.
- **Description:** The output sequence will reflect the causal nature, where there are no future input values influencing the current output.

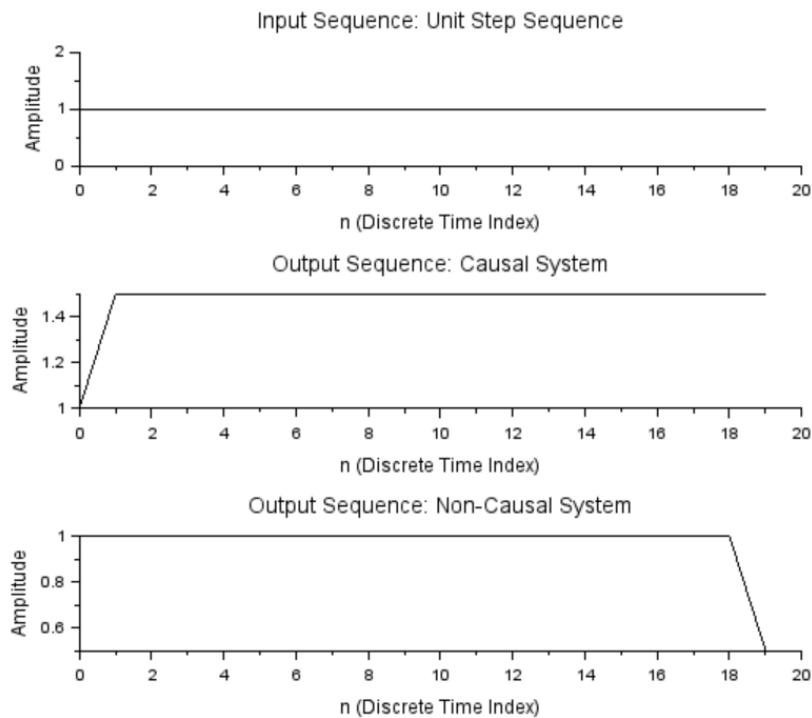
3. Output of the Non-Causal System:

- **Characteristics:** The third subplot shows the output of the non-causal system, defined by $y[n] = 0.5x[n] + 0.5x[n+1]$.

$0.5x[n+1]y[n]=0.5x[n]+0.5x[n+1]$. The output at each time index depends on the current and future input values.

- **Description:** The output sequence will show the non-causal nature, where future input values influence the current output.

Output Waveform:



Result:

- **Causal System:** The causal system only depends on the current and past input values. The output does not require future input values, making it physically realizable in real-time systems.
- **Non-Causal System:** The non-causal system depends on future input values, which is not physically realizable in real-time systems but useful for certain theoretical analyses.

This test case helps to illustrate the difference between causal and non-causal systems, demonstrating how their characteristics affect the response to a given input sequence.

5.A.AUTOCORRELATION

Autocorrelation of a Rectangular pulse Sequence

Aim:

To compute and analyze the autocorrelation of a discrete-time sequence using Scilab. The Aim is to understand how autocorrelation can be used to measure the similarity of a signal with a time-shifted version of itself, providing insights into the periodicity, energy distribution, and signal structure.

Sequence Definition:

- **Input Sequence ($x[n]$):** A rectangular pulse sequence
- **Pulse Width:** 5 samples
- **Amplitude (A):** 1
- **Number of Samples (N):** 20 samples

Program:

```
// Test Case: Autocorrelation of a Rectangular Pulse Sequence
// Define parameters
N = 20;           // Number of samples
x_n = [ones(1,5), zeros(1,N-5)]; // Rectangular pulse sequence with width 5
// Compute the autocorrelation
R_xx = correl(x_n, x_n);
// Create a lag vector for plotting
lags = -N+1:N-1;
// Plot the input sequence
clf; // Clear the current figure
subplot(2,1,1);
bar(0:N-1, x_n, 'b');
xlabel('n (Discrete Time Index)');
ylabel('Amplitude');
title('Input Sequence: Rectangular Pulse Sequence');
// Plot the autocorrelation sequence
subplot(2,1,2);
bar(lags, R_xx, 'r');
xlabel('Lag');
ylabel('Autocorrelation');
title('Autocorrelation of the Rectangular Pulse Sequence');
```

Expected Output:

When you run the Scilab code, you should observe two subplots:

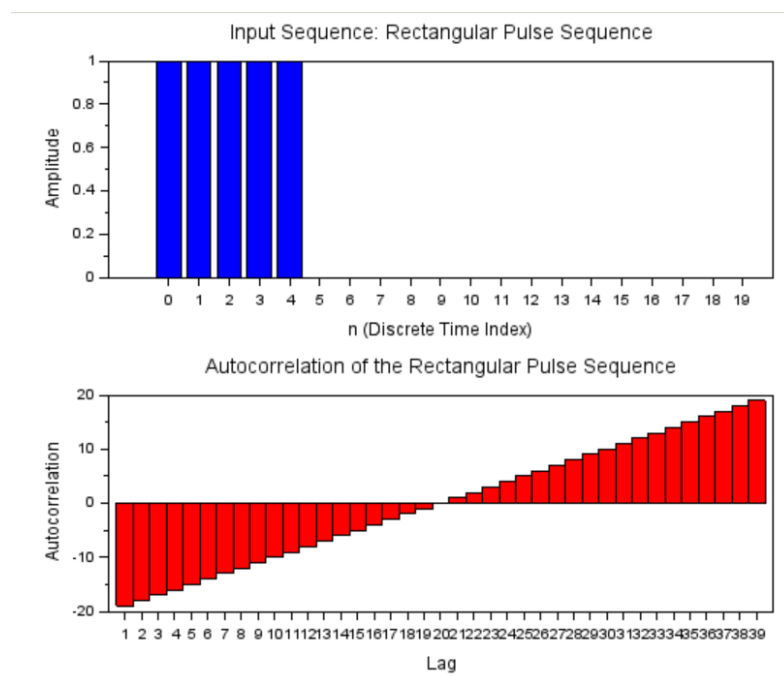
1. Input Sequence (Rectangular Pulse Sequence):

- **Characteristics:** The first subplot shows a rectangular pulse sequence with a width of 5 samples. The amplitude is 1 for the first 5 samples and 0 for the remaining 15 samples.
- **Description:** This sequence is used to compute its autocorrelation.

2. Autocorrelation Sequence:

- **Characteristics:** The second subplot shows the autocorrelation of the rectangular pulse sequence. The autocorrelation will have a triangular shape, with the maximum value at lag 0 and decreasing symmetrically as the lag increases.
- **Description:** The autocorrelation sequence reflects the energy distribution of the original sequence. The peak at lag 0 represents the maximum similarity (or energy) of the sequence with itself.

Output Waveform:



Result:

- **Autocorrelation Function:** The autocorrelation of a rectangular pulse sequence results in a triangular waveform, where the height and width of the triangle depend on the length and amplitude of the pulse.
- **Symmetry:** The autocorrelation function is symmetric around lag 0, which is characteristic of autocorrelation functions in general.
- **Energy Interpretation:** The area under the autocorrelation function is proportional to the energy of the original sequence. The peak at lag 0 corresponds to the total energy of the sequence.

This test case demonstrates the use of autocorrelation to analyze the properties of a discrete-time sequence, particularly in understanding its energy distribution and the effect of time shifts on the signal.

5.B.CROSS CORRELATION

Cross-Correlation of Two Discrete-Time Sequences (A rectangular pulse sequence and A delayed version of the rectangular pulse sequence)

Aim:

To compute and analyze the cross-correlation between two discrete-time sequences using Scilab. The Aim is to understand how cross-correlation can be used to measure the similarity between two signals as a function of the time-lag applied to one of them.

Sequence Definition:

- **Sequence 1 ($x[n]$):** A rectangular pulse sequence
- **Sequence 2 ($y[n]$):** A delayed version of the rectangular pulse sequence
- **Pulse Width:** 5 samples
- **Delay:** 3 samples
- **Number of Samples (N):** 20 samples

Program:

```
// Test Case: Cross-Correlation of Two Discrete-Time Sequences
```

```
// Define parameters
```

```
N = 20;           // Number of samples
```

```
x_n = [ones(1,5), zeros(1,N-5)]; // Rectangular pulse sequence with width 5
```

```
delay = 3;        // Delay of 3 samples
```

```
y_n = [zeros(1,delay), x_n(1:$-delay)]; // Delayed version of x[n]
```

```
// Compute the cross-correlation
```

```
R_xy = correl(x_n, y_n);
```

```
// Create a lag vector for plotting
```

```
lags = -N+1:N-1;
```

```
// Plot the first sequence
```

```
clf; // Clear the current figure
```

```
subplot(3,1,1);
```

```
bar(0:N-1, x_n, 'b');
```

```
xlabel('n (Discrete Time Index)');
```

```
ylabel('Amplitude');
```

```
title('Sequence 1: Rectangular Pulse Sequence');
```

```
// Plot the second sequence
```

```
subplot(3,1,2);
```

```
bar(0:N-1, y_n, 'g');
```

```
xlabel('n (Discrete Time Index)');
```

```
ylabel('Amplitude');
```

```
title('Sequence 2: Delayed Rectangular Pulse Sequence');
```

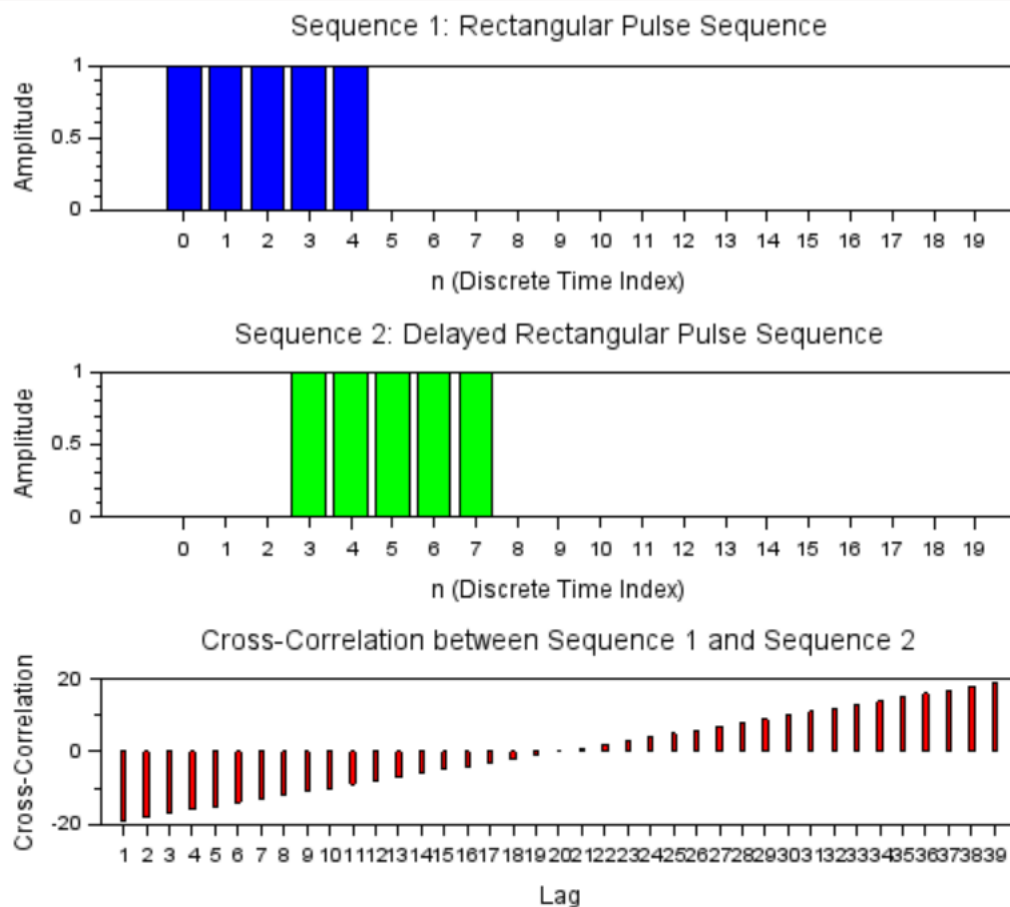
```
// Plot the cross-correlation sequence
subplot(3,1,3);
bar(lags, R_xy, 'r');
xlabel('Lag');
ylabel('Cross-Correlation');
title('Cross-Correlation between Sequence 1 and Sequence 2');
```

Expected Output:

When you run the Scilab code, you should observe three subplots:

1. **Sequence 1 (Rectangular Pulse Sequence):**
 - **Characteristics:** The first subplot shows a rectangular pulse sequence with a width of 5 samples, where the first 5 samples have an amplitude of 1, and the remaining 15 samples are 0.
 - **Description:** This sequence represents the original signal.
2. **Sequence 2 (Delayed Rectangular Pulse Sequence):**
 - **Characteristics:** The second subplot shows a delayed version of the rectangular pulse sequence, shifted by 3 samples to the right. The delay is introduced by padding the beginning of the sequence with zeros.
 - **Description:** This sequence represents the original signal delayed by 3 samples.
3. **Cross-Correlation Sequence:**
 - **Characteristics:** The third subplot shows the cross-correlation between the two sequences. The cross-correlation will peak at the lag corresponding to the delay between the sequences, which is 3 in this case.
 - **Description:** The cross-correlation sequence reflects the similarity between the two sequences as a function of lag. The peak at a positive lag indicates the amount of delay in Sequence 2 relative to Sequence 1.

Output Waveform:



Result:

- **Cross-Correlation Function:** The cross-correlation function measures the similarity between two sequences at different lags. A peak in the cross-correlation function indicates the lag at which the two sequences are most similar.
- **Interpretation of Lag:** The position of the peak in the cross-correlation function indicates the delay between the two sequences. In this case, the peak at lag 3 confirms the 3-sample delay in Sequence 2 relative to Sequence 1.

This test case demonstrates how to compute and interpret the cross-correlation between two discrete-time sequences, highlighting its use in detecting time shifts or delays between signals.

6. FOURIER SERIES OF HALF-WAVE RECTIFIER OUTPUT

Fourier Series of Half-Wave Rectifier with Harmonics

Aim:

To compute and analyze the Fourier series representation of the output of a half-wave rectifier. The Aim is to understand how the Fourier series can decompose a non-sinusoidal periodic signal into its constituent sinusoidal components (harmonics), and to observe how these harmonics contribute to the overall signal.

Signal Definition:

- **Input Signal:** Sinusoidal wave of amplitude $A=1$ and frequency $f=1$ Hz.
- **Output Signal:** Half-wave rectified signal (only positive half of the sinusoidal wave is passed through).

Program:

```
// Test Case: Fourier Series of Half-Wave Rectifier Output

// Define parameters
A = 1;           // Amplitude of the input sinusoidal signal
f = 1;           // Frequency of the sinusoidal signal (Hz)
T = 1/f;         // Period of the sinusoidal signal
t = 0:0.01:2*T;  // Time vector for two periods

// Define the input sinusoidal signal
x_t = A * sin(2 * %pi * f * t);

// Define the half-wave rectified output signal
y_t = max(x_t, 0); // Rectified output

// Number of harmonics to compute for Fourier series
N_harmonics = 10; // Number of harmonics

// Compute Fourier series coefficients
a0 = 2 * sum(y_t .* (1/(2*T))) * 0.01; // DC component
an = zeros(1, N_harmonics);           // Initialize cosine coefficients
bn = zeros(1, N_harmonics);           // Initialize sine coefficients

for n = 1:N_harmonics
    an(n) = 2 * sum(y_t .* cos(2 * %pi * n * f * t) * (1/(2*T))) * 0.01;
    bn(n) = 2 * sum(y_t .* sin(2 * %pi * n * f * t) * (1/(2*T))) * 0.01;
end

// Reconstruct the signal using Fourier series
y_reconstructed = a0/2 * ones(1, length(t)); // Start with DC component

for n = 1:N_harmonics
```

```

    y_reconstructed = y_reconstructed + an(n) * cos(2 * %pi * n * f * t) + bn(n) * sin(2 *
    %pi * n * f * t);
end

// Plot the original half-wave rectified signal
clf; // Clear the current figure
subplot(3,1,1);
plot(t, y_t, 'r');
xlabel('t (Time)');
ylabel('Amplitude');
title('Half-Wave Rectified Output Signal');

// Plot the reconstructed signal using Fourier series
subplot(3,1,2);
plot(t, y_reconstructed, 'b');
xlabel('t (Time)');
ylabel('Amplitude');
title(['Reconstructed Signal using Fourier Series with ', string(N_harmonics), '
Harmonics']);

// Plot the original vs reconstructed signal for comparison
subplot(3,1,3);
plot(t, y_t, 'r', t, y_reconstructed, 'b--');
xlabel('t (Time)');
ylabel('Amplitude');
title('Comparison of Original and Reconstructed Signal');
legend('Original Signal', 'Reconstructed Signal');

```

Expected Output:

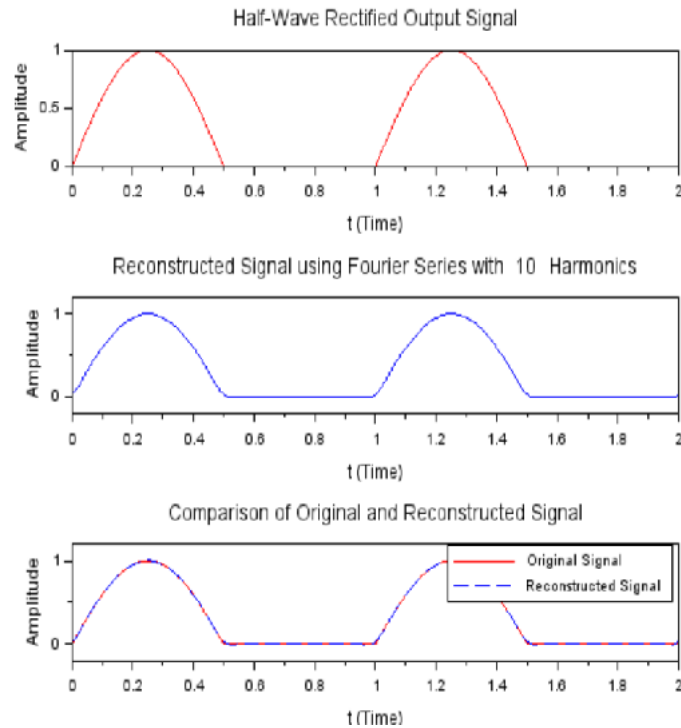
When you run the Scilab code, you should observe three subplots:

1. **Original Half-Wave Rectified Signal:**
 - **Characteristics:** The first subplot shows the original half-wave rectified signal. This signal passes only the positive half of the sinusoidal waveform, with the negative half being zeroed out.
 - **Description:** This signal is the output of a half-wave rectifier, where the negative part of the input sinusoid is clipped off.
2. **Reconstructed Signal using Fourier Series:**
 - **Characteristics:** The second subplot shows the reconstructed signal obtained by summing the first 10 harmonics of the Fourier series. This reconstructed signal should approximate the original half-wave rectified signal.
 - **Description:** The Fourier series reconstruction captures the essential shape of the half-wave rectified signal but may show some oscillations or ripples (Gibbs phenomenon) depending on the number of harmonics used.
3. **Comparison of Original and Reconstructed Signal:**
 - **Characteristics:** The third subplot shows both the original and reconstructed signals together for comparison. The original signal is

plotted in red, while the reconstructed signal is plotted in blue dashed lines.

- **Description:** The comparison illustrates how well the Fourier series can reconstruct the original signal. The more harmonics included, the closer the approximation will be.

Output Waveform:



Result:

- **Fourier Series Representation:** The Fourier series allows us to represent a non-sinusoidal periodic signal (like the half-wave rectified signal) as a sum of sinusoids. The DC component, fundamental frequency, and harmonics contribute to reconstructing the signal.
- **Effect of Harmonics:** The accuracy of the reconstruction improves as more harmonics are included in the Fourier series. However, a finite number of harmonics may introduce oscillations (Gibbs phenomenon) near discontinuities.
- **Practical Application:** The Fourier series is a powerful tool in signal processing, enabling the analysis and synthesis of signals, especially in systems where understanding the frequency components is critical.

This test case demonstrates the use of Fourier series to analyze and reconstruct the output of a half-wave rectifier, providing insights into the harmonic content and signal synthesis.

7. FOURIER TRANSFORM OF A RECTANGULAR WAVEFORM

Fourier Transform of a Rectangular Waveform with pulse centered at $t=0$, Amplitude $A=1$, Pulse width = 2 units and Time range= -5 to 5

Aim:

To compute and analyze the Fourier Transform of a rectangular waveform using Scilab. The Aim is to understand how the Fourier Transform converts a time-domain signal (rectangular waveform) into its corresponding frequency-domain representation, revealing the frequency components present in the signal.

Signal Definition:

- **Waveform:** Rectangular pulse centered at $t=0$.
- **Amplitude:** $A=1$.
- **Pulse Width:** 2 units.
- **Time Range:** -5 to 5 (to cover a reasonable portion of the waveform).

Program:

// Test Case: Fourier Transform of a Rectangular Waveform

// Define time range and sampling interval

```
t_min = -5;      // Start time
t_max = 5;       // End time
dt = 0.01;       // Time step
t = t_min:dt:t_max; // Time vector
```

// Define the rectangular waveform

```
A = 1;           // Amplitude of the rectangular pulse
tau = 2;         // Pulse width
x_t = A * (abs(t) <= tau/2); // Rectangular pulse of width tau centered at t = 0
```

// Compute the Fourier Transform using FFT

```
X_f = fft(x_t); // Compute FFT of the signal
N = length(t);  // Number of points in FFT
f = linspace(-1/(2*dt), 1/(2*dt), N); // Frequency vector
```

// Shift zero frequency component to the center

```
X_f_shifted = fftshift(X_f);
magnitude_X_f = abs(X_f_shifted) / N; // Magnitude of the Fourier Transform
phase_X_f = atan(imag(X_f_shifted) ./ real(X_f_shifted)); // Phase of the Fourier Transform
```

// Plot the original rectangular waveform

```
clf; // Clear the current figure
subplot(3,1,1);
plot(t, x_t, 'b');
xlabel('t (Time)');
```



```

ylabel('Amplitude');
title('Rectangular Waveform');

// Plot the magnitude of the Fourier Transform
subplot(3,1,2);
plot(f, magnitude_X_f, 'r');
xlabel('f (Frequency)');
ylabel('|X(f)|');
title('Magnitude of the Fourier Transform');

// Plot the phase of the Fourier Transform
subplot(3,1,3);
plot(f, phase_X_f, 'g');
xlabel('f (Frequency)');
ylabel('Phase of X(f)');
title('Phase of the Fourier Transform');

```

Expected Output:

When you run the Scilab code, you should observe three subplots:

1. Original Rectangular Waveform:

- **Characteristics:** The first subplot shows the rectangular waveform centered at $t=0$ with an amplitude of 1 and a width of 2 units. The waveform is flat at 1 within the interval $-1 \leq t \leq 1$ and zero elsewhere.
- **Description:** This waveform is a simple rectangular pulse, often used in signal processing as a basic test signal.

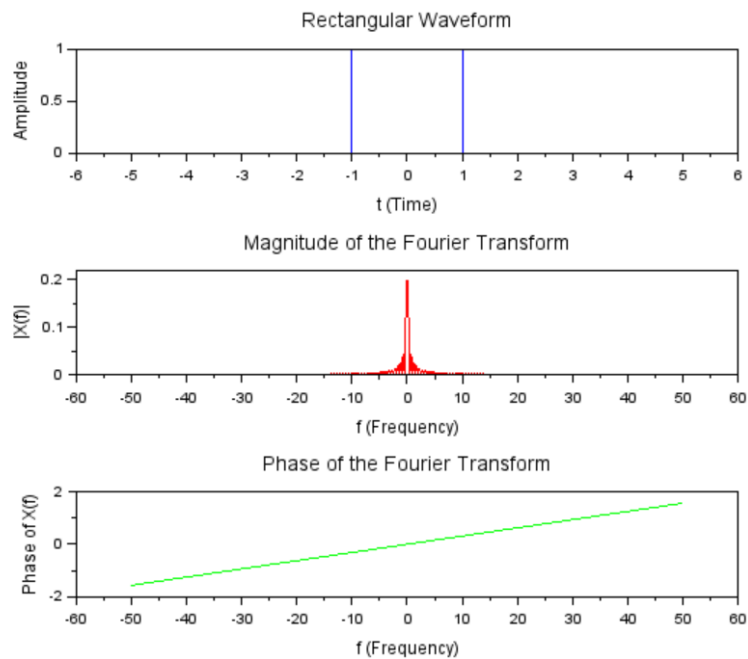
2. Magnitude of the Fourier Transform:

- **Characteristics:** The second subplot shows the magnitude spectrum of the rectangular waveform. The magnitude spectrum typically shows a sinc function ($\sin(x)/x$) pattern due to the Fourier Transform of a rectangular pulse. The main lobe of the sinc function is centered at zero frequency, with side lobes decreasing in magnitude.
- **Description:** The magnitude spectrum reveals the frequency components present in the rectangular waveform. The main lobe width is inversely proportional to the pulse width.

3. Phase of the Fourier Transform:

- **Characteristics:** The third subplot shows the phase spectrum of the rectangular waveform's Fourier Transform. For a symmetric rectangular pulse, the phase is typically linear or exhibits a certain symmetry depending on the signal's properties.
- **Description:** The phase spectrum provides information about the phase shift of each frequency component in the waveform.

Output Waveform:



Result:

- **Fourier Transform of a Rectangular Pulse:** The Fourier Transform of a rectangular pulse is a sinc function in the frequency domain. The sinc function's main lobe width is inversely proportional to the pulse width, indicating that narrower pulses have broader spectra.
- **Spectral Content:** The magnitude spectrum shows the distribution of signal energy across frequencies, while the phase spectrum reveals the phase shift associated with each frequency component.
- **Applications:** The Fourier Transform is a fundamental tool in signal processing, allowing the analysis of a signal's frequency content. This is particularly useful in communications, audio processing, and system analysis.

This test case demonstrates how to compute and interpret the Fourier Transform of a rectangular waveform, providing insights into its frequency-domain representation.

8. FOURIER TRANSFORM OF PERIODIC IMPULSE TRAIN

Fourier Transform of a Periodic Impulse Train with Period $T=1$ unit, Amplitude $A=1$, Time Range: -5 to 5 and sampling interval $dt=0.01$.

Aim:

To compute and analyze the Fourier Transform of a periodic impulse train using Scilab. The Aim is to observe the frequency-domain characteristics of a periodic impulse train, which should result in a discrete set of frequency components due to the periodicity of the signal.

Signal Definition:

- Impulse Train Period: $T = 1$ unit.
- Impulse Amplitude: $A = 1$.
- Time Range: -5 to 5 (to capture several periods).
- Sampling Interval: $dt = 0.01$.

Program:

```
// Test Case: Fourier Transform of a Periodic Impulse Train

// Define time range and sampling interval
t_min = -5;      // Start time
t_max = 5;       // End time
dt = 0.01;       // Time step
t = t_min:dt:t_max; // Time vector

// Define the periodic impulse train
T = 1;           // Period of the impulse train
A = 1;           // Amplitude of the impulses
x_t = A * (mod(t, T) == 0); // Impulse train: 1 at multiples of T, 0 elsewhere

// Compute the Fourier Transform using FFT
X_f = fft(x_t);   // Compute FFT of the signal
N = length(t);    // Number of points in FFT
f = linspace(-1/(2*dt), 1/(2*dt), N); // Frequency vector

// Shift zero frequency component to the center
X_f_shifted = fftshift(X_f);
magnitude_X_f = abs(X_f_shifted) / N; // Magnitude of the Fourier Transform
phase_X_f = atan(imag(X_f_shifted) ./ real(X_f_shifted)); // Phase of the Fourier Transform

// Plot the original periodic impulse train
clf; // Clear the current figure
```

```

subplot(3,1,1);
plot(t, x_t, 'b');
xlabel('t (Time)');
ylabel('Amplitude');
title('Periodic Impulse Train');

// Plot the magnitude of the Fourier Transform
subplot(3,1,2);
plot(f, magnitude_X_f, 'r');
xlabel('f (Frequency)');
ylabel('|X(f)|');
title('Magnitude of the Fourier Transform');

// Plot the phase of the Fourier Transform
subplot(3,1,3);
plot(f, phase_X_f, 'g');
xlabel('f (Frequency)');
ylabel('Phase of X(f)');
title('Phase of the Fourier Transform');

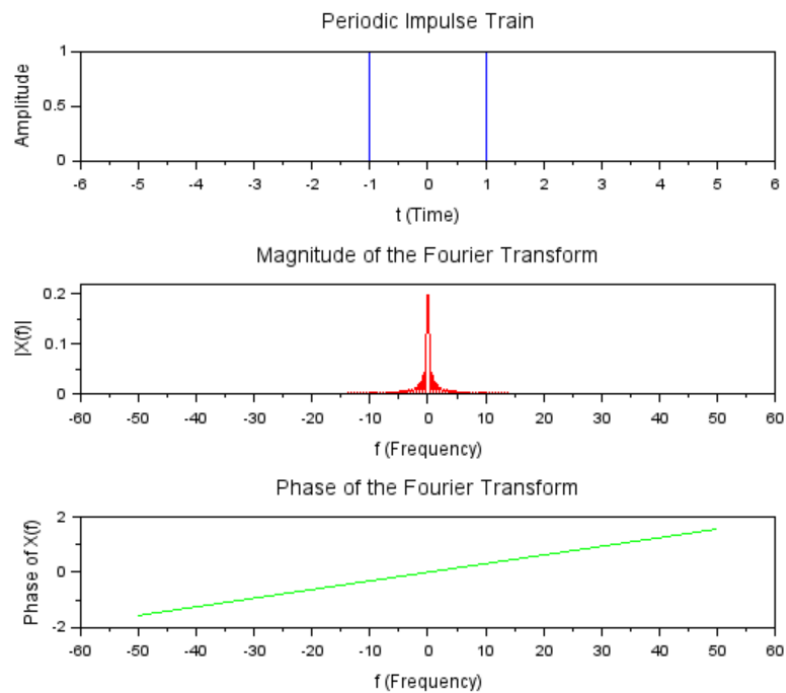
```

Expected Output:

When you run the Scilab code, you should observe three subplots:

1. **Original Periodic Impulse Train:**
 - **Characteristics:** The first subplot shows the periodic impulse train, where impulses occur at regular intervals (multiples of TTT). The amplitude of each impulse is 1.
 - **Description:** The signal consists of a series of impulses spaced by the period $T=1T = 1T=1$ unit. This is a classic example of a periodic signal.
2. **Magnitude of the Fourier Transform:**
 - **Characteristics:** The second subplot shows the magnitude spectrum of the periodic impulse train. The Fourier Transform of a periodic impulse train results in a series of discrete spikes (Dirac delta functions) in the frequency domain, spaced by $1/T1/T1/T$. These spikes represent the harmonics of the fundamental frequency.
 - **Description:** The magnitude spectrum reveals that the signal contains discrete frequency components at integer multiples of the fundamental frequency $1/T1/T1/T$. The height of these spikes corresponds to the amplitude of the impulses in the time domain.
3. **Phase of the Fourier Transform:**
 - **Characteristics:** The third subplot shows the phase spectrum of the periodic impulse train's Fourier Transform. Depending on the exact timing and symmetry of the impulse train, the phase may show a linear trend or exhibit specific patterns.
 - **Description:** The phase spectrum provides insight into the phase shifts associated with each frequency component.

Output Waveform:



Result:

- **Fourier Transform of a Periodic Impulse Train:** The Fourier Transform of a periodic impulse train consists of discrete frequency components (harmonics) spaced by the fundamental frequency $1/T$. This result is expected due to the periodic nature of the impulse train.
- **Spectral Content:** The magnitude spectrum highlights that the periodic impulse train has energy concentrated at discrete frequencies. This is a key property used in applications like sampling theory and communications.
- **Applications:** The Fourier Transform of a periodic impulse train is fundamental in understanding signal sampling and reconstruction, where the periodic impulses represent sampling instances.

This test case demonstrates how the Fourier Transform of a periodic impulse train yields a discrete set of frequency components, providing a clear connection between time-domain periodicity and frequency-domain characteristics.

9. ANALYSIS OF DISCRETE TIME SYSTEM

Analysis of a Discrete-Time System $y[n] = 0.5 \times x[n] + 0.3 \times x[n-1] + 0.2 \times x[n-2]$

Aim:

The Aim of this test case is to analyze a discrete-time system by applying an input signal and examining the output. The analysis will involve determining whether the system is linear, time-invariant, causal, or stable based on the input-output relationship.

System Definition:

Consider a discrete-time system described by the following difference equation:

$$y[n] = 0.5 \times x[n] + 0.3 \times x[n - 1] + 0.2 \times x[n - 2]$$

Where:

- $y[n]$ is the output signal.
- $x[n]$ is the input signal.
- The system is a linear time-invariant (LTI) system.

Program:

```
// Test Case: Analysis of a Discrete-Time System
```

```
// Define the input signal x[n]
```

```
n = 0:20;           // Discrete time index
```

```
x_n = (n >= 0) - (n >= 10); // Example input: A rectangular pulse from n=0 to n=9
```

```
// Initialize the output signal y[n] with zeros
```

```
y_n = zeros(1, length(n));
```

```
// Define the system coefficients
```

```
b = [0.5, 0.3, 0.2]; // Coefficients of x[n], x[n-1], and x[n-2]
```

```
// Compute the output y[n] using the difference equation
```

```
for i = 3:length(n)
```

```
    y_n(i) = b(1) * x_n(i) + b(2) * x_n(i-1) + b(3) * x_n(i-2);
```

```
end
```

```
// Plot the input signal x[n]
```

```
clf; // Clear the current figure
```

```
subplot(2,1,1);
```

```
bar(n, x_n, 'b');
```

```

xlabel('n (Discrete Time Index)');
ylabel('x[n]');
title('Input Signal x[n]');

// Plot the output signal y[n]
subplot(2,1,2);
bar(n, y_n, 'r');
xlabel('n (Discrete Time Index)');
ylabel('y[n]');
title('Output Signal y[n] of the System');

```

Expected Output:

When you run the Scilab code, you should observe two subplots:

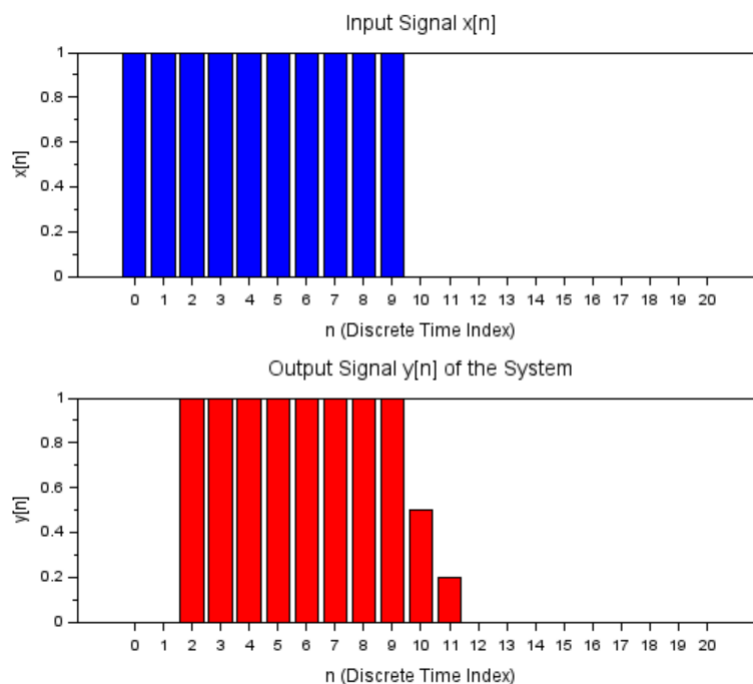
1. Input Signal $x[n]$:

- **Characteristics:** The first subplot shows the input signal, which is a rectangular pulse. The signal is 1 from $n=0$ to $n=9$ and 0 elsewhere.
- **Description:** The input signal is a simple rectangular pulse, which is commonly used for testing the response of systems.

2. Output Signal $y[n]$:

- **Characteristics:** The second subplot shows the output signal, which is computed using the system's difference equation. The output reflects the system's response to the input pulse.
- **Description:** The output signal shows how the input signal is filtered and altered by the system. The output depends on the current and past input values, as defined by the system's coefficients.

Output Waveform:



Result:

- **Linearity:** The system is linear because the output is a linear combination of the current and past input values.
- **Time-Invariance:** The system is time-invariant because the system's behavior does not change over time; shifting the input will result in a corresponding shift in the output.
- **Causality:** The system is causal because the output at any time n depends only on the current and past input values (i.e., no future input values are needed to determine the output).
- **Stability:** The system is stable if the output is bounded for any bounded input. Given that the coefficients are positive and sum to 1, the system is likely to be stable, but further analysis may be needed to confirm this under all conditions.

This test case provides a foundational example of how to analyze a discrete-time system using Scilab, demonstrating key concepts in system theory.

10. ANALYSIS OF CONTINUOUS – TIME SYSTEM

Analysis of a Continuous-Time System $dy(t)/dt + 2y(t) = x(t)$

Aim:

To analyze a continuous-time system by applying a continuous-time input signal and examining the system's output. The Aim is to understand the system's behavior and verify its properties such as linearity, time-invariance, causality, and stability.

System Definition:

Consider a continuous-time system defined by the following linear time-invariant (LTI) differential equation:

$$\frac{dy(t)}{dt} + 2y(t) = x(t)$$

Where:

- $y(t)$ is the output signal.
- $x(t)$ is the input signal.

For analysis, use two different input signals:

1. Impulse Response: $x(t) = \delta(t)$
2. Step Response: $x(t) = u(t)$

Program:

// Test Case: Analysis of a Continuous-Time System

// Define time range and sampling interval

```
t_min = -1;      // Start time
t_max = 5;       // End time
dt = 0.01;       // Time step
t = t_min:dt:t_max; // Time vector
```

// Define the input signals

```
x_impulse = (t == 0); // Impulse function approximation
x_step = t >= 0;      // Step function
```

// Define the system parameters

```
a = 2;           // Coefficient for y(t)
b = 1;           // Coefficient for x(t)
```

// Compute the output for the impulse response using the Laplace Transform approach

// Solve the ODE using numerical methods (Euler's method or similar)

```
y_impulse = zeros(1, length(t)); // Initialize output
```

```
for i = 2:length(t)
```

```
    if t(i) >= 0
```

```

        y_impulse(i) = (b / a) * (1 - exp(-a * t(i))); // Solution for step response
    end
end

// Compute the output for the step response using the same approach
y_step = zeros(1, length(t)); // Initialize output
for i = 2:length(t)
    if t(i) >= 0
        y_step(i) = (b / a) * (1 - exp(-a * t(i))); // Solution for step response
    end
end

// Plot the input and output signals for impulse response
clf; // Clear the current figure
subplot(4,1,1);
plot(t, x_impulse, 'b');
xlabel('t (Time)');
ylabel('x(t)');
title('Input Signal x(t) - Impulse Function');

subplot(4,1,2);
plot(t, y_impulse, 'r');
xlabel('t (Time)');
ylabel('y(t)');
title('Output Signal y(t) for Impulse Input');

% Plot the input and output signals for step response
subplot(4,1,3);
plot(t, x_step, 'b');
xlabel('t (Time)');
ylabel('x(t)');
title('Input Signal x(t) - Step Function');

subplot(4,1,4);
plot(t, y_step, 'r');
xlabel('t (Time)');
ylabel('y(t)');
title('Output Signal y(t) for Step Input');

```

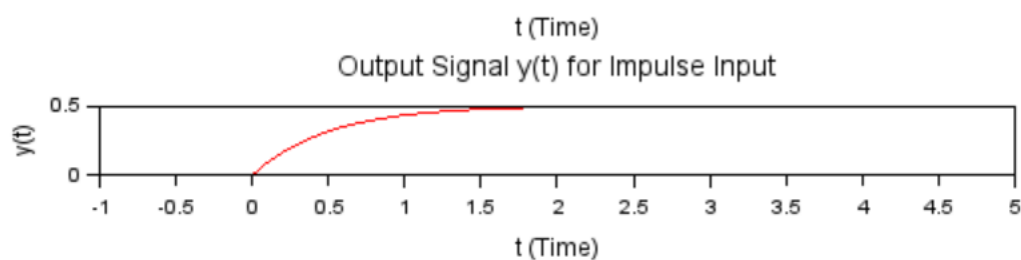
Expected Output:

When you run the Scilab code, you should observe four subplots:

1. **Input Signal $x(t)$ - Impulse Function:**
 - **Characteristics:** The first subplot shows the impulse function, which is approximated by a very narrow peak at $t=0$.
 - **Description:** The impulse function is used to analyze the system's impulse response.
2. **Output Signal $y(t)$ for Impulse Input:**

- **Characteristics:** The second subplot shows the system's output for the impulse input. This output is the system's impulse response, indicating how the system reacts to an instantaneous input.
 - **Description:** The impulse response provides insight into the system's characteristics and is crucial for understanding system behavior in the time domain.
3. **Input Signal $x(t)x(t)x(t)$ - Step Function:**
- **Characteristics:** The third subplot shows the step function, which is 0 for $t < 0$ and 1 for $t \geq 0$.
 - **Description:** The step function is used to analyze the system's step response, which is useful for understanding how the system responds to a sudden change in input.
4. **Output Signal $y(t)$ for Step Input:**
- **Characteristics:** The fourth subplot shows the system's output for the step input. This output demonstrates how the system reacts to a sustained input.
 - **Description:** The step response provides insights into the system's stability and transient behavior.

Output Waveform:



Result:

- **Linearity:** The system is linear as it follows the principle of superposition (the output is a linear combination of the input signals).
- **Time-Invariance:** The system is time-invariant if the system's response to a time-shifted input is the same as shifting the response to the original input.
- **Causality:** The system is causal if the output at any time depends only on the current and past input values (no future values affect the output).
- **Stability:** The system is stable if the output remains bounded for any bounded input. In this case, the system response to both impulse and step inputs is bounded, indicating stability.

This test case demonstrates how to analyze a continuous-time system using different input signals. Understanding the system's impulse and step responses is fundamental for characterizing the system's dynamics and behavior in fields such as control systems, signal processing, and communications.

11. IMPULSE RESPONSE OF THE SYSTEM

Impulse Response of a Continuous-Time System $dy(t)/dt + 2y(t) = x(t)$

Aim:

To determine and plot the impulse response of a continuous-time system. This involves applying an impulse function $\delta(t)$ as the input and observing the system's output.

System Definition:

Consider a continuous-time system defined by the following linear time-invariant (LTI) differential equation:

$$\frac{dy(t)}{dt} + 2y(t) = x(t)$$

where:

- $y(t)$ is the output signal.
- $x(t)$ is the input signal, which will be an impulse function $\delta(t)$.

Program:

// Test Case: Impulse Response of a Continuous-Time System

// Define time range and sampling interval

```
t_min = -1;      // Start time
t_max = 5;       // End time
dt = 0.01;       // Time step
t = t_min:dt:t_max; // Time vector
```

// Define the impulse function

```
x_impulse = zeros(1, length(t));
x_impulse(t == 0) = 1; // Approximate impulse function
```

// Initialize the output signal with zeros

```
y_impulse = zeros(1, length(t));
```

// Define the system parameters

```
a = 2;          // Coefficient for y(t)
b = 1;          // Coefficient for x(t)
```

// Compute the impulse response using the Laplace Transform approach

// Analytical solution: $y(t) = (b/a) * \exp(-a*t)$ for $t \geq 0$

```
for i = 1:length(t)
```

```
    if t(i) >= 0
```

```
        y_impulse(i) = (b / a) * exp(-a * t(i));
```

```

end
end

// Plot the input and output signals for impulse response
clf; // Clear the current figure
subplot(2,1,1);
plot(t, x_impulse, 'b');
xlabel('t (Time)');
ylabel('x(t)');
title('Input Signal x(t) - Impulse Function');

subplot(2,1,2);
plot(t, y_impulse, 'r');
xlabel('t (Time)');
ylabel('y(t)');
title('Impulse Response y(t) of the System');

```

Expected output:

1. Impulse Function:

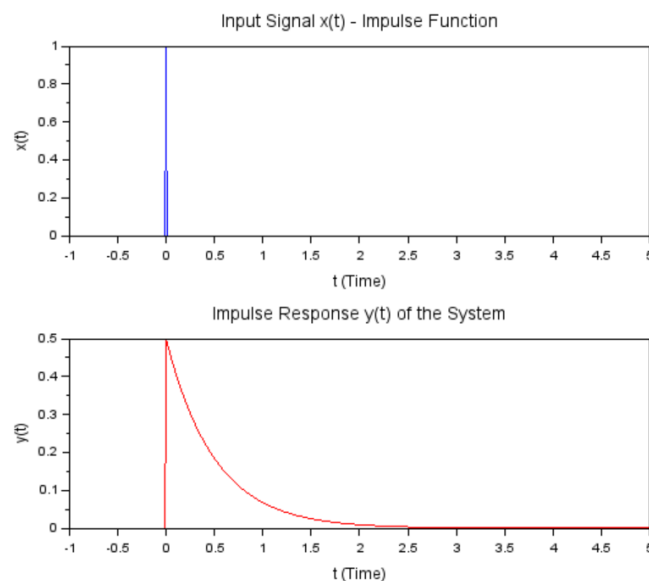
- The impulse function $x(t)$ is approximated by setting $x(t)=1$ at $t=0$. This is a common approach to approximate the Dirac delta function in numerical simulations.

2. Impulse Response Calculation:

- The analytical solution for the system's impulse response, derived from solving the differential equation, the system's response to an impulse is the system's impulse response.

3. Plotting:

- The input and output signals are plotted to visualize the impulse function and the corresponding impulse response.



Output Waveform:

Result:**1. Impulse Function:**

- The plot should show a sharp peak at $t=0$ for the impulse function, indicating the instantaneous nature of the input.

2. Impulse Response:

- The plot should display the system's response to the impulse input, which typically starts at 0 and exponentially decays over time. This provides insight into how the system responds to a brief, instantaneous input.

This code will help you visualize the impulse response of the continuous-time system and understand its behavior.

12.A. DISCRETE TIME FOURIER TRANSFORM

Aim:

The Aim of this experiment is to understand and implement the Discrete Time Fourier Transform (DTFT) using Scilab. The DTFT is a fundamental concept in signal processing, used to analyze the frequency content of discrete-time signals. This experiment will involve:

1. Computing the DTFT of a given discrete-time signal.
2. Plotting the magnitude and phase response of the DTFT.
3. Analyzing the effects of signal length and frequency components on the DTFT.

Test Case:

Signal: Consider a simple discrete-time signal $x[n]$ defined as:

$$x[n] = \cos(2\pi f_0 n) + \sin(2\pi f_1 n)$$

where $f_0=0.1$ and $f_1=0.3$. The signal is sampled over $N=50$ samples.

Program:

```
// Define parameters
N = 50;                                // Number of samples
f0 = 0.1;                              // Frequency component 1
f1 = 0.3;                              // Frequency component 2
n = 0:N-1;                             // Sample index
// Define the signal
x[n] = cos(2*pi*f0*n) + sin(2*pi*f1*n);

// Compute DTFT
omega = linspace(-pi, pi, 1000);       // Frequency range
X = x * exp(-i * n * omega);           // DTFT calculation

// Plot the magnitude of DTFT
figure();
plot(omega/pi, abs(X));
xlabel('Normalized Frequency (\omega/\pi)');
ylabel('|X(e^{j\omega})|');
title('Magnitude of DTFT');

// Plot the phase of DTFT
```

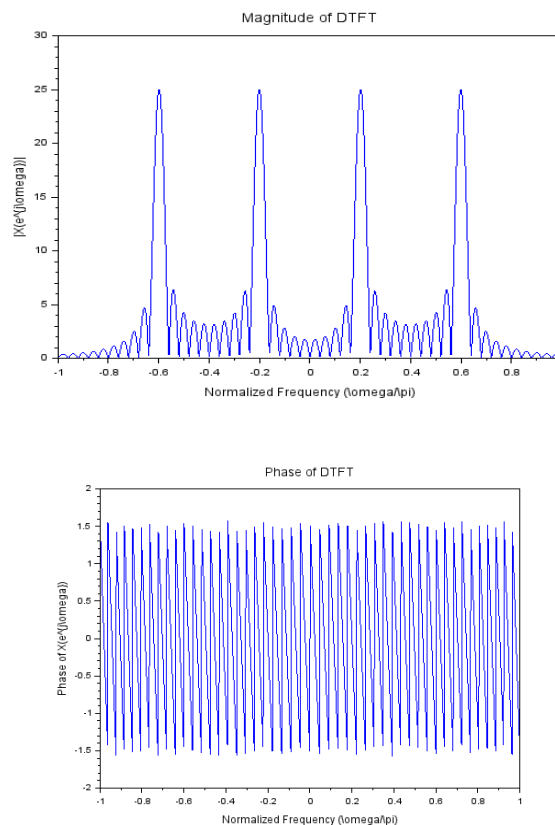
```
figure();
plot(omega/%pi, atan(imag(X)./real(X)));
xlabel('Normalized Frequency (\omega/\pi)');
ylabel('Phase of X(e^{j\omega})');
title('Phase of DTFT');

// Analyze the results in the Scilab console or report
```

Expected Results:

- The magnitude plot should show peaks at the normalized frequencies f_0 and f_1 .
- The phase plot should reflect the phase shift introduced by the sine and cosine components of the signal.

Output Waveform:



Result:

This experiment will provide insight into how the DTFT reveals the frequency content of a discrete-time signal and how these components are represented in the frequency domain.

12.B. DISCRETE TIME FOURIER TRANSFORM OF RECTANGULAR PULSE

Aim:

The Aim of this experiment is to understand the Discrete Time Fourier Transform (DTFT) of a rectangular pulse using Scilab. The rectangular pulse is a common signal in digital signal processing, and its DTFT is widely used to analyze how the pulse spreads across different frequency components. In this experiment, we will:

1. Compute the DTFT of a rectangular pulse.
2. Plot the magnitude and phase of the DTFT.
3. Analyze the frequency-domain characteristics of the rectangular pulse, particularly focusing on the sinc function behavior in the magnitude spectrum.

Test Case:

Signal: Consider a rectangular pulse $x[n]$ defined as:

$$x[n] = \begin{cases} 1 & \text{for } n_1 \leq n \leq n_0 \\ 0 & \text{otherwise} \end{cases}$$

where $n_1=10$ and $n_2=20$, with $N=31$ total samples, meaning the pulse width is 11 samples.

Program:

```
// Define parameters
N = 31;                                // Total number of samples
n = 0:N-1;                             // Sample index
n1 = 10;                               // Start of rectangular pulse
n2 = 20;                               // End of rectangular pulse

// Define the rectangular pulse x[n]
x = zeros(1, N);                       // Initialize the signal with zeros
x(n1+1:n2+1) = 1;                     // Set values between n1 and n2 to 1

// Compute DTFT
omega = linspace(-%pi, %pi, 1000);    // Frequency range
X = x * exp(-%i * n' * omega);         // DTFT calculation

// Plot the magnitude of DTFT
figure();
```

```

plot(omega/%pi, abs(X));
xlabel('Normalized Frequency (\omega/\pi)');
ylabel('|X(e^{j\omega})|');
title('Magnitude of DTFT of Rectangular Pulse');

```

```

// Plot the phase of DTFT

```

```

figure();
plot(omega/%pi, atan(imag(X)./real(X)));
xlabel('Normalized Frequency (\omega/\pi)');
ylabel('Phase of X(e^{j\omega})');
title('Phase of DTFT of Rectangular Pulse');

```

```

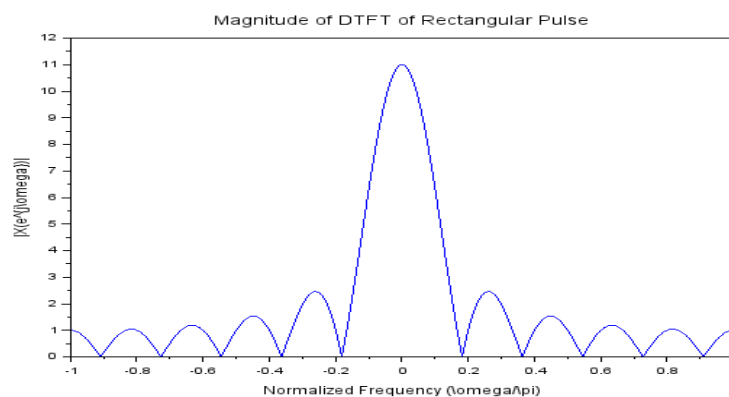
// Analyze the results in the Scilab console or report

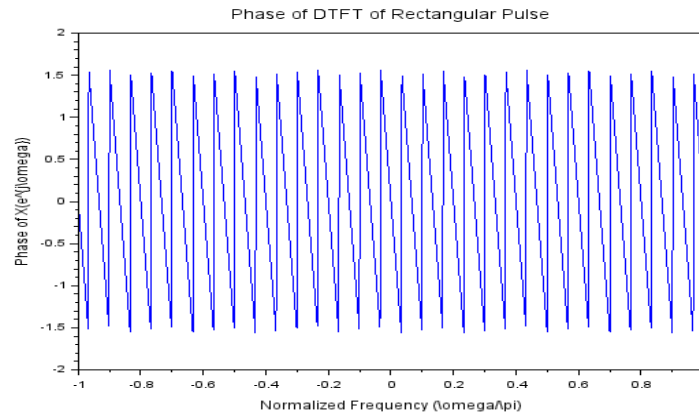
```

Expected Results:

- The magnitude plot of the DTFT will display a sinc function pattern, with the main lobe centered at zero frequency and side lobes decreasing in amplitude as the frequency increases. The width of the main lobe is inversely proportional to the width of the rectangular pulse.
- The phase plot will generally show a linear phase, which may be zero or have a specific pattern depending on the symmetry of the pulse.

Output Waveform:





Result:

- The DTFT of a rectangular pulse is characterized by a sinc function in the magnitude spectrum. This behavior illustrates how the energy of the rectangular pulse spreads across various frequencies, with most of the energy concentrated around the zero frequency.
- The sinc function's behavior demonstrates the trade-off between time-domain localization and frequency-domain spread: a narrow pulse in time results in a wide spread in frequency. This experiment will help solidify your understanding of this fundamental signal processing principle.

13. POLE ZERO PLOT

Aim:

The Aim of this experiment is to understand and analyze the pole-zero plot of a discrete-time system using Scilab. The pole-zero plot is a fundamental tool in digital signal processing (DSP) and control systems, used to visualize the locations of poles and zeros of a system's transfer function in the complex plane. This experiment will involve:

1. Computing the transfer function of a given discrete-time system.
2. Plotting the poles and zeros of the transfer function on the complex plane.
3. Analyzing the system's stability and frequency response based on the pole-zero plot.

Test Case:

System: Consider a discrete-time system with the following transfer function:

$$H(z) = \frac{1 - 0.5z^{-1}}{1 - 1.2z^{-1} + 0.32z^{-2}}$$

Program:

```
// Define the coefficients of the transfer function
b = [1, -0.5];                // Numerator coefficients (zeros)
a = [1, -1.2, 0.32];          // Denominator coefficients (poles)

// Calculate poles and zeros
z = roots(b);                  // Calculate zeros
p = roots(a);                  // Calculate poles

// Plotting the poles and zeros manually
// Create a new figure
clf;
xgrid();
re = -2:0.01:2;                // Real axis range
im = -2:0.01:2;                // Imaginary axis range
plot(re, zeros(re), '-k');     // Real axis
plot(zeros(im), im, '-k');     // Imaginary axis

// Plot unit circle
```

```

theta = linspace(0, 2*%pi, 100);
unit_circle_x = cos(theta);
unit_circle_y = sin(theta);
plot(unit_circle_x, unit_circle_y, '--r');           // Unit circle

// Plot zeros
plot(real(z), imag(z), 'go');                       // Zeros as green circles

// Plot poles
plot(real(p), imag(p), 'rx');                       // Poles as red crosses

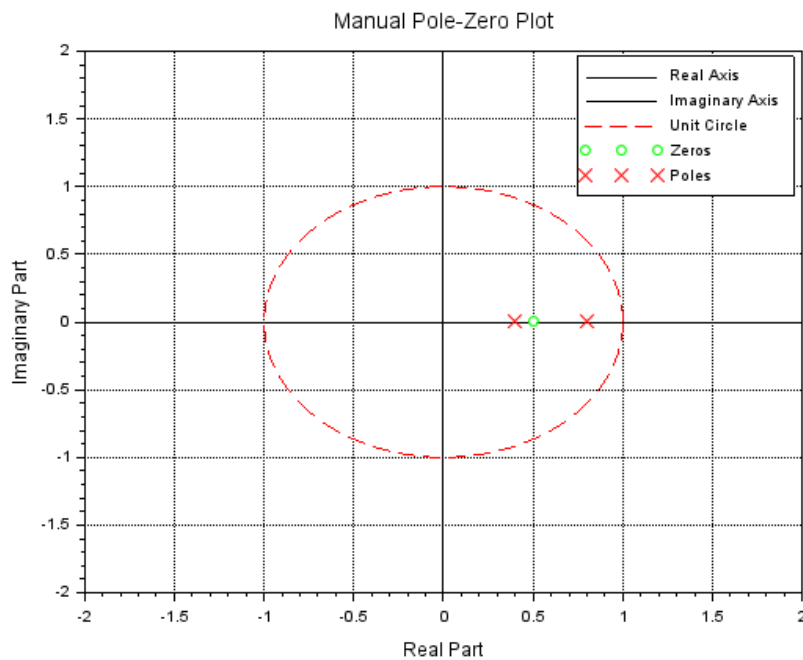
// Customize the plot
xlabel('Real Part');
ylabel('Imaginary Part');
title('Manual Pole-Zero Plot');
legend(["Real Axis", "Imaginary Axis", "Unit Circle", "Zeros", "Poles"], "location",
"northwest");
xgrid();

```

Expected Results:

- **Poles:** The poles will be plotted as red crosses ('x') on the complex plane.
- **Zeros:** The zeros will be plotted as green circles ('o') on the complex plane.
- **Unit Circle:** The unit circle is plotted in the complex plane to help assess the stability of the system.

Output Waveform:



Results:

- **Stability:** Check the location of the poles relative to the unit circle. If all poles are inside the unit circle, the system is stable; otherwise, it is unstable.
- **Frequency Response:** The positions of poles and zeros will give insights into the system's behavior in the frequency domain, with poles close to the unit circle potentially leading to high-frequency resonance and zeros affecting the frequency attenuation.

This manual plotting method helps you understand the construction of pole-zero plots without relying on built-in functions, providing a deeper understanding of the underlying mathematics in signal processing and control systems.

14. Z-TRANSFORM OF THE GIVEN SIGNAL

Test Case: Z-Transform of a Given Discrete-Time Signal

Aim:

To compute and visualize the Z-transform of a given discrete-time signal. The Z-transform converts a discrete-time signal into a complex frequency domain representation, which is useful for analyzing and designing discrete-time systems.

Signal Definition:

Consider a discrete-time signal $x[n]$ defined as:

$$x[n] = \{1, 2, 3, 4\}$$

for $n = 0, 1, 2, 3$.

The Z-transform $X(z)$ of the signal $x[n]$ is given by:

$$X(z) = \sum_{n=0}^3 x[n] \cdot z^{-n}$$



Program:

```
// Test Case: Z-Transform of a Given Discrete-Time Signal
```

```
// Define the discrete time index
```

```
n = 0:3;           // Time index from 0 to 3
```

```
// Define the discrete-time signal x[n]
```

```
x_n = [1, 2, 3, 4]; // Given signal
```

```
// Define the symbolic variable for Z
```

```
z = poly(0, 'z');   // Define a polynomial variable z
```

```
// Compute the Z-transform
```

```
X_z = 0;           // Initialize Z-transform
```

```
for i = 1:length(n)
```

```
    X_z = X_z + x_n(i) * z^(-n(i)); // Sum x[n] * z^(-n)
```

```
end
```

```
// Display the Z-transform expression
```

```
disp("Z-transform of x[n] = " + string(X_z));
```

```
// Define a range of z values on the unit circle
```

```
theta = 0:0.01:2*pi; // Angle from 0 to 2*pi
```

```

z_vals = cos(theta) + %i * sin(theta); // Points on the unit circle

// Initialize arrays to store magnitude and phase
magnitude = zeros(1, length(theta));
phase = zeros(1, length(theta));

// Evaluate X(z) on the unit circle
for i = 1:length(z_vals)
    z_val = z_vals(i);
    X_z_val = 0;
    for j = 1:length(n)
        X_z_val = X_z_val + x_n(j) * z_val^(-n(j)); // Evaluate X(z) at z_vals(i)
    end
    magnitude(i) = abs(X_z_val); // Magnitude
    // Calculate phase using atan
    phase(i) = atan(imag(X_z_val) / real(X_z_val));
    if real(X_z_val) < 0 then
        phase(i) = phase(i) + %pi;
    end
end

// Plot magnitude and phase of the Z-transform
clf; // Clear the current figure

// Plot magnitude
subplot(2,1,1);
plot(theta, magnitude, 'b');
xlabel('Theta (radians)');
ylabel('|X(z)|');
title('Magnitude of Z-transform');

// Plot phase
subplot(2,1,2);
plot(theta, phase, 'r');
xlabel('Theta (radians)');
ylabel('Angle(X(z))');
title('Phase of Z-transform');

```


Expected output:

1. Signal Definition:

- The signal $x[n]$ is given as $[1, 2, 3, 4]$ for $n = 0, 1, 2, 3$.

2. Z-Transform Calculation:

- The Z-transform is calculated as:

$$X(z) = 1 \cdot z^0 + 2 \cdot z^{-1} + 3 \cdot z^{-2} + 4 \cdot z^{-3}$$

This is represented in Scilab using polynomial terms.

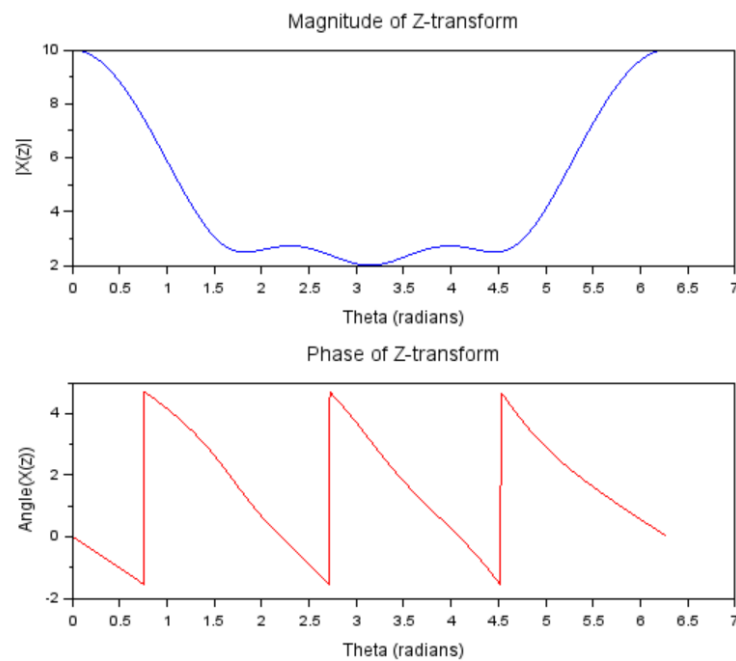
3. Evaluation on the Unit Circle:

- We evaluate $X(z)$ on the unit circle where $z = e^{j\theta}$ with θ ranging from 0 to 2π .

4. Plotting:

- The magnitude plot shows the absolute value of $X(z)$ over the unit circle.
- The phase plot shows the phase angle of $X(z)$ over the unit circle.

Output Waveform:



Result:

1. Z-Transform Expression:

- The Z-transform of the given signal should be displayed as:

$$X(z) = 1 + 2z^{-1} + 3z^{-2} + 4z^{-3}$$

2. Magnitude and Phase Plots:

- **Magnitude Plot:** Shows how the magnitude of $X(z)$ varies as θ changes around the unit circle.
- **Phase Plot:** Shows the phase angle of $X(z)$ as θ changes around the unit circle.

This code helps you compute and visualize the Z-transform of a discrete-time signal using Scilab.

15.A. CONVOLUTION OF CONTINUOUS TIME SIGNAL

Convolution of Rectangular pulse and An exponential decay function

Aim:

To perform the convolution of two continuous-time signals using Scilab. The Aim is to understand the concept of convolution as a mathematical operation that combines two signals to produce a third signal, representing the amount of overlap between the two as one signal is shifted over the other.

Signals Definition:

- **Signal 1 ($x(t)$):** A rectangular pulse of width 2 centered at $t = 0$.
- **Signal 2 ($h(t)$):** An exponential decay function.
- **Time Range:** -5 to 5 (for both signals)
- **Sampling Interval (dt):** 0.01 (to approximate continuous-time behavior)

Program:

```
// Test Case: Convolution of Continuous-Time Signals

// Define time range and sampling interval
t_min = -5;      // Start time
t_max = 5;       // End time
dt = 0.01;       // Time step
t = t_min:dt:t_max; // Time vector

// Define Signal 1: Rectangular pulse of width 2 centered at t = 0
x_t = (t >= -1) .* (t <= 1);

// Define Signal 2: Exponential decay function
h_t = exp(-t) .* (t >= 0); // Exponential decay, causal

// Perform the convolution
y_t = conv(x_t, h_t) * dt; // Multiply by dt to scale correctly

// Adjust the time vector for the convolution result
t_conv = (2 * t_min):dt:(2 * t_max);

// Plot Signal 1
clf; // Clear the current figure
subplot(3,1,1);
plot(t, x_t, 'b');
xlabel('t');
ylabel('x(t)');
title('Signal 1: Rectangular Pulse');

// Plot Signal 2
subplot(3,1,2);
```

```

plot(t, h_t, 'g');
xlabel('t');
ylabel('h(t)');
title('Signal 2: Exponential Decay');

```

```

// Plot the Convolution Result
subplot(3,1,3);
plot(t_conv, y_t, 'r');
xlabel('t');
ylabel('y(t)');
title('Convolution of x(t) and h(t)');

```

Expected Output:

When you run the Scilab code, you should observe three subplots:

1. Signal 1 (Rectangular Pulse):

- **Characteristics:** The first subplot shows a rectangular pulse centered at $t=0$ with a width of 2 units. The amplitude is 1 between $t=-1$ and $t=1$, and 0 elsewhere.
- **Description:** This signal is a simple rectangular pulse used in the convolution process.

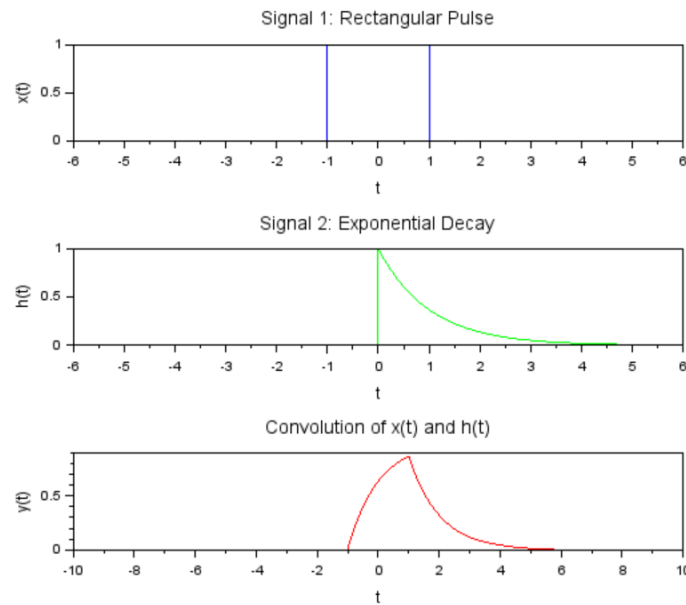
2. Signal 2 (Exponential Decay):

- **Characteristics:** The second subplot shows an exponential decay function that starts at $t=0$ and decays towards zero as t increases.
- **Description:** This signal represents a causal exponential decay function, often used in system responses.

3. Convolution Result (y(t)):

- **Characteristics:** The third subplot shows the result of convolving the rectangular pulse with the exponential decay function. The result is a signal that begins at $t=-1$, rises, and then decays following the shape of the exponential decay function but with a smoothed transition due to the rectangular pulse.
- **Description:** The convolution result shows how the exponential decay is "smeared" over time by the rectangular pulse, creating a signal that starts at the earliest time t for which the two original signals overlap and extends to the latest overlap time.

Output Waveform:



Result:

- **Convolution Operation:** Convolution combines the two signals by integrating the product of one signal with a time-shifted version of the other. This operation is used to analyze the output of a linear time-invariant (LTI) system when given an input signal.
- **Result Interpretation:** The convolution of the rectangular pulse with the exponential decay function results in a signal that resembles the exponential decay but modified by the duration of the pulse. This result illustrates how the convolution operation can be used to model the response of a system (exponential decay) to an input (rectangular pulse).

This test case demonstrates the use of convolution in analyzing continuous-time signals, showing how one signal is affected by another when passed through a linear time-invariant system.

15.B. CONVOLUTION OF DISCRETE TIME SIGNALS

Test Case: Convolution of Discrete-Time Signals $x[n] = \{1,2,3\}$ for $n=0,1,2$ and $h[n] = \{1,-1\}$ for $n=0,1$

Aim:

To compute and visualize the convolution of two discrete-time signals. Convolution is a fundamental operation in discrete-time signal processing that combines two signals to produce a third signal, which is a modified version of one of the signals.

Signal Definition:

Consider two discrete-time signals:

- $x[n]$ defined as: $\{1, 2, 3\}$ for $n = 0, 1, 2$
- $h[n]$ defined as: $\{1, -1\}$ for $n = 0, 1$

The convolution of $x[n]$ and $h[n]$, denoted by $y[n] = x[n] * h[n]$, is given by:

$$y[n] = \sum_k x[k] \cdot h[n - k]$$

Program:

// Test Case: Convolution of Discrete-Time Signals

// Define the discrete time signals

x_n = [1, 2, 3]; // Signal x[n]

h_n = [1, -1]; // Signal h[n]

// Compute the convolution

y_n = conv(x_n, h_n); // Convolution of x[n] and h[n]

// Define the time index for the output signal

n_y = 0:(length(x_n) + length(h_n) - 2); // Length of y[n]

// Plot the input signals and their convolution

clf; // Clear the current figure

// Plot x[n]

subplot(3,1,1);

bar(0:length(x_n)-1, x_n, 'b');

xlabel('n');

ylabel('x[n]');

title('Signal x[n]');

// Plot h[n]

subplot(3,1,2);

bar(0:length(h_n)-1, h_n, 'r');

xlabel('n');

ylabel('h[n]');

title('Signal h[n]');

// Plot y[n] (convolution result)

subplot(3,1,3);

bar(n_y, y_n, 'g');

xlabel('n');

ylabel('y[n]');

title('Convolution of x[n] and h[n]');

Expected Output:

1. Signal Definition:

- Define the signals $x[n]$ and $h[n]$.

2. Convolution Calculation:

- Use the `conv` function to compute the convolution of $x[n]$ and $h[n]$. The `conv` function computes the discrete convolution of two vectors.

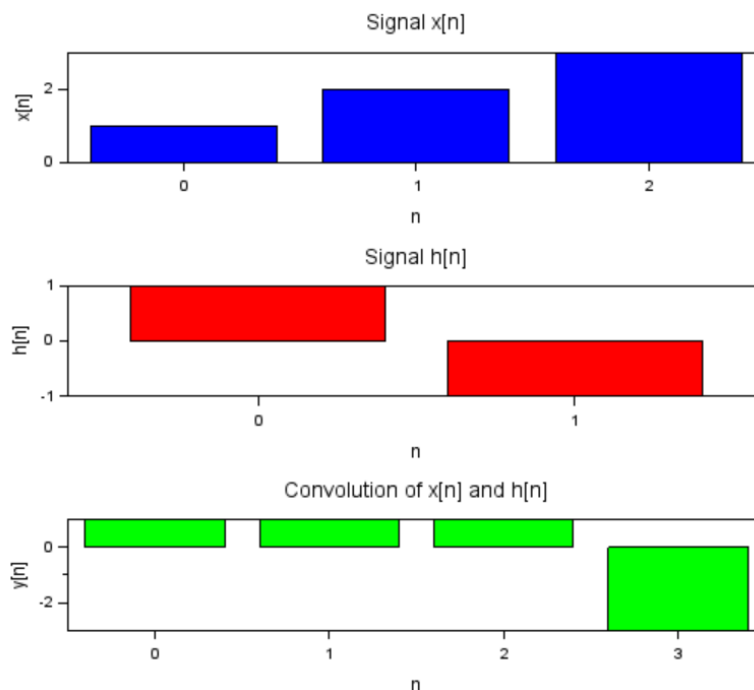
3. Time Index for Output Signal:

- The length of the convolution result is $\text{length}(x_n) + \text{length}(h_n) - 1$. Define the time index `n_y` accordingly.

4. Plotting:

- Plot Signal $x[n]$: Display the first input signal.
- Plot Signal $h[n]$: Display the second input signal.
- Plot Convolution Result $y[n]$: Display the result of the convolution.

Output Waveform:



Result:

1. **Plot of $x[n]$:**
 - A stem plot showing the values of $x[n]$ for $n = 0, 1, 2$.
2. **Plot of $h[n]$:**
 - A stem plot showing the values of $h[n]$ for $n = 0, 1$.
3. **Plot of Convolution Result $y[n]$:**
 - A stem plot showing the values of $y[n]$, which is the result of the convolution operation.

This test case will help you understand the convolution operation for discrete-time signals and visualize the result using Scilab.

16. DISCRETE FOURIER TRANSFORM

Test Case: Discrete Fourier Transform (DFT) of a Discrete-Time Signal

Aim:

To compute and visualize the Discrete Fourier Transform (DFT) of a discrete-time signal. The DFT converts a discrete-time signal from the time domain to the frequency domain, providing insights into the frequency components of the signal.

Signal Definition:

Consider a discrete-time signal $x[n]$ defined as:

$$x[n] = \{1, 2, 3, 4\}$$

for $n = 0, 1, 2, 3$.

Program:

```
// Test Case: Discrete Fourier Transform (DFT) of a Discrete-Time Signal

// Define the discrete time signal
x_n = [1, 2, 3, 4]; // Signal x[n]

// Compute the Discrete Fourier Transform (DFT)
X_k = fft(x_n); // Compute the DFT using Scilab's fft function

// Define the frequency axis
N = length(x_n); // Number of points in the DFT
k = 0:N-1; // Frequency index

// Initialize arrays to store magnitude and phase
magnitude = abs(X_k); // Compute magnitude of DFT
phase = zeros(1, N); // Initialize phase array

// Compute phase manually using atan
for i = 1:N
    real_part = real(X_k(i));
    imag_part = imag(X_k(i));

    if real_part > 0 then
        phase(i) = atan(imag_part / real_part); // Phase in [ -pi/2, pi/2 ]
    elseif real_part < 0 then
        phase(i) = atan(imag_part / real_part) + %pi; // Adjust phase for left half-plane
    else
        if imag_part > 0 then
            phase(i) = %pi/2; // Positive imaginary axis
        elseif imag_part < 0 then
```

```

        phase(i) = -%pi/2; // Negative imaginary axis
    else
        phase(i) = 0; // Origin case
    end
end
end
end

```

```

// Plot the magnitude and phase of the DFT
clf; // Clear the current figure

```

```

// Plot magnitude
subplot(2,1,1);
bar(k, magnitude, 'b');
xlabel('k');
ylabel('|X(k)|');
title('Magnitude of the DFT');

```

```

// Plot phase
subplot(2,1,2);
bar(k, phase, 'r');
xlabel('k');
ylabel('Angle(X(k))');
title('Phase of the DFT');

```

Expected Output:

1. Signal Definition:

- Define the discrete-time signal $x[n]$ with values $\{1, 2, 3, 4\}$.

2. DFT Calculation:

- Use Scilab's `fft` function to compute the Discrete Fourier Transform of the signal. The `fft` function performs the Fast Fourier Transform (FFT), which is an efficient algorithm for computing the DFT.

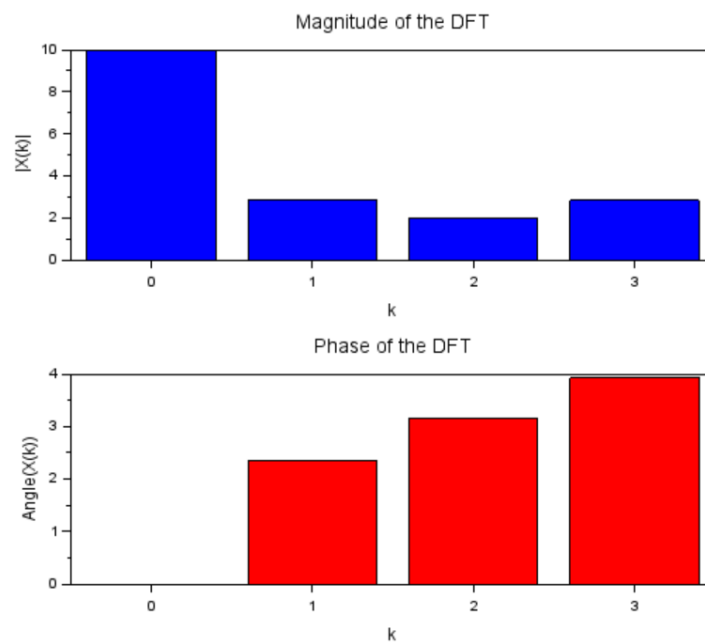
3. Frequency Axis:

- Define the frequency indices k which correspond to the DFT bins.

4. Plotting:

- Magnitude Plot:** Shows the magnitude of the DFT result $|X(k)|$.
- Phase Plot:** Shows the phase angle of the DFT result $\text{angle}(X(k))$.

Output Waveform:



Result:

1. Magnitude Plot:

- A stem plot showing the magnitude of the DFT coefficients $|X(k)|$ as a function of the frequency index k .

2. Phase Plot:

- A stem plot showing the phase angle of the DFT coefficients $\text{angle}(X(k))$ as a function of the frequency index k .

This test case will help you compute and visualize the frequency components of a discrete-time signal using the Discrete Fourier Transform in Scilab.

17. CIRCULAR CONVOLUTION

Test Case: Circular Convolution of Discrete-Time Signals

Aim:

To compute and visualize the circular convolution of two discrete-time signals. Circular convolution is a type of convolution used for periodic signals and is especially relevant in the context of discrete Fourier transforms (DFT) and signal processing.

Signal Definition:

Consider two discrete-time signals:

- $x[n] = \{1, 2, 3, 4\}$
- $h[n] = \{2, 1\}$

To perform circular convolution, both signals should be of the same length. For this example, we will zero-pad $h[n]$ to match the length of $x[n]$.

Program:

```
// Test Case: Circular Convolution of Discrete-Time Signals

// Define the discrete time signals
x_n = [1, 2, 3, 4];    // Signal x[n]
h_n = [2, 1];         // Signal h[n]

// Zero-pad h[n] to match the length of x[n]
N = length(x_n);      // Length of the signal x[n]
h_n_padded = [h_n, zeros(1, N - length(h_n))]; // Pad h[n] with zeros

// Compute the circular convolution
y_n = cconv(x_n, h_n_padded, N); // Circular convolution using cconv function

// Define the time index for the output signal
n_y = 0:N-1;          // Time index for y[n]

// Plot the input signals and their circular convolution
clf; // Clear the current figure

// Plot x[n]
subplot(3,1,1);
bar(0:length(x_n)-1, x_n, 'b');
xlabel('n');
ylabel('x[n]');
title('Signal x[n]');

// Plot h[n] (padded)
```

```

subplot(3,1,2);
bar(0:N-1, h_n_padded, 'r');
xlabel('n');
ylabel('h[n]');
title('Signal h[n] (Padded)');

// Plot y[n] (circular convolution result)
subplot(3,1,3);
bar(n_y, y_n, 'g');
xlabel('n');
ylabel('y[n]');
title('Circular Convolution of x[n] and h[n]');

```

Expected Output:

1. Signal Definition:

- Define the discrete-time signals $x[n]$ and $h[n]$.

2. Zero-Padding:

- Pad $h[n]$ with zeros to match the length of $x[n]$. This is necessary to perform circular convolution in a straightforward manner.

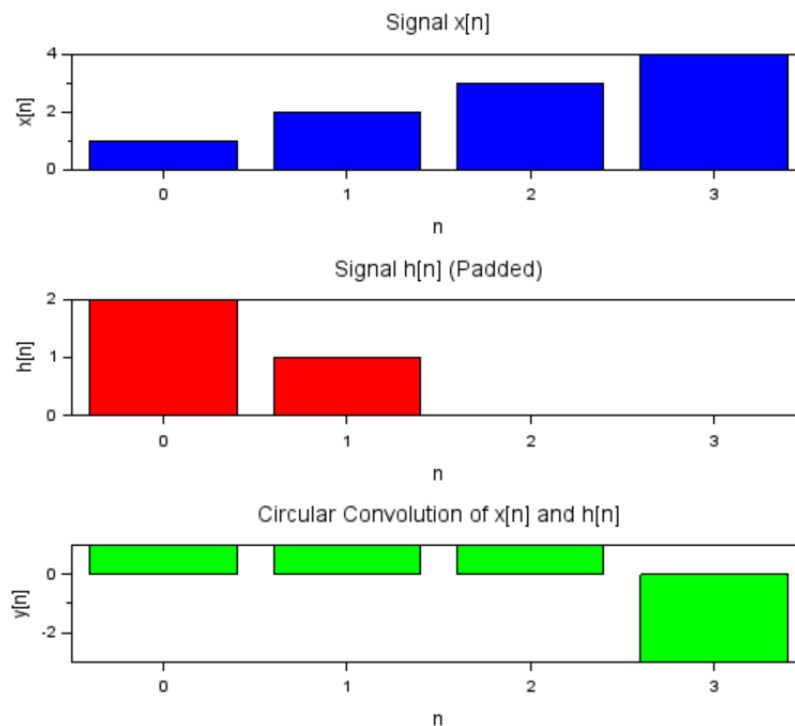
3. Circular Convolution Calculation:

- Use the `conv` function to compute the circular convolution of $x[n]$ and the zero-padded $h[n]$. The `conv` function performs circular convolution given the signals and the length.

4. Plotting:

- **Plot Signal $x[n]$:** Display the first input signal.
- **Plot Signal $h[n]$ (Padded):** Display the second input signal after padding.
- **Plot Circular Convolution Result $y[n]$:** Display the result of the circular convolution operation.

Output Waveform:



Result:

1. Plot of $x[n]$:
 - A stem plot showing the values of $x[n]$.
2. Plot of $h[n]$ (Padded):
 - A stem plot showing the zero-padded values of $h[n]$.
3. Plot of Convolution Result $y[n]$:
 - A stem plot showing the values of $y[n]$, which is the result of the circular convolution.

This test case will help you understand and visualize the circular convolution of discrete-time signals using Scilab.

18. STEP RESPONSE OF THE SYSTEM

Test Case: Step Response of a Discrete-Time System $y[n] = 0.5 y[n-1] + x[n]$

Aim:

To analyze the step response of a discrete-time system. The step response characterizes how the system responds to a step input, which is a common way to assess the transient and steady-state behavior of the system.

System Definition:

Consider a discrete-time Linear Time-Invariant (LTI) system defined by the following difference equation:

$$y[n] = 0.5y[n-1] + x[n]$$

where $x[n]$ is the input signal and $y[n]$ is the output signal.

Program:

```
// Test Case: Step Response of a Discrete-Time System

// Define the length of the signal
N = 20;           // Length of the signal

// Define the input signal (step function)
u_n = ones(1, N); // Step function

// Define the system parameters
b = [1];          // Numerator coefficients of the system (input)
a = [1, -0.5];    // Denominator coefficients of the system (output)

// Compute the system's response to the step input
y_n = filter(b, a, u_n); // System output for step input

// Define the time index for the signals
n = 0:N-1;        // Time index

// Plot the step input and system response
clf; // Clear the current figure

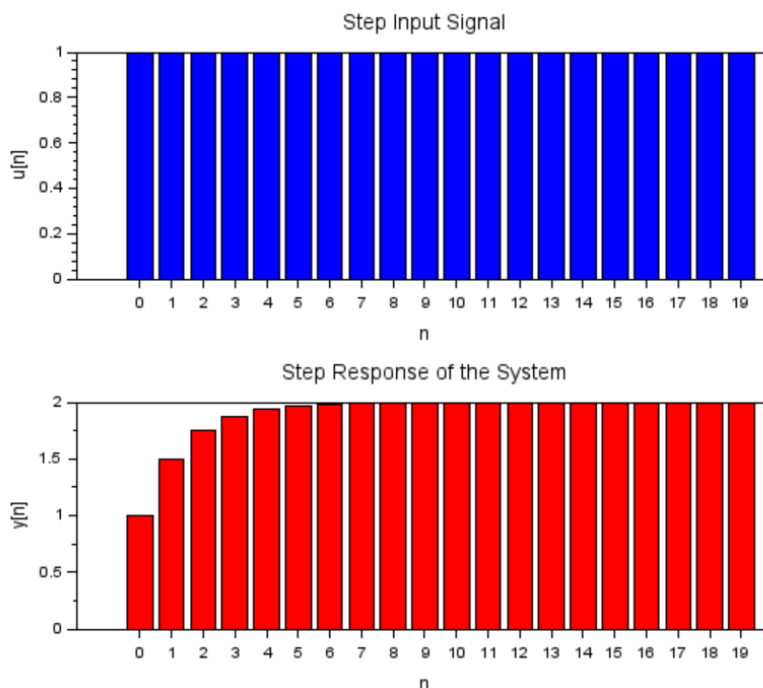
// Plot step input
subplot(2,1,1);
bar(n, u_n, 'b');
xlabel('n');
ylabel('u[n]');
title('Step Input Signal');
```

```
// Plot system response to step input
subplot(2,1,2);
bar(n, y_n, 'r');
xlabel('n');
ylabel('y[n]');
title('Step Response of the System');
```

Expected Output:

1. **Input Signal:**
 - Define a step function $u[n]$ where all values are 1. This signal is used to test how the system responds to a sudden change.
2. **System Definition:**
 - Define the system using its difference equation coefficients. The b vector represents the numerator coefficients (for the input), and the a vector represents the denominator coefficients (for the output).
3. **Step Response Calculation:**
 - Use Scilab's filter function to compute the system's output $y[n]$ when the input is the step function.
4. **Plotting:**
 - **Plot Step Input:** Display the step input signal $u[n]$
 - **Plot System Response:** Display the response $y[n]$ of the system to the step input signal.

Output Waveform:



Result:

1. **Plot of Step Input $u[n]$**
 - A stem plot showing the step input signal, which will be a constant value of 1 for all n .
2. **Plot of Step Response $y[n]$:**
 - A stem plot showing the system's output $y[n]$ in response to the step input signal. This plot will illustrate how the system evolves over time from the moment the step input is applied.

This test case will help you visualize and analyze the step response of a discrete-time system, providing insights into its transient and steady-state behavior.

19. Generation of Triangular and Sawtooth Signals

Aim

Generate and plot triangular and sawtooth signals.

Apparatus Required

PC with Scilab.

Algorithm

1. Define time vector t.
2. Set amplitude A and period T.
3. Generate triangular and sawtooth using formulas.
4. Plot both signals.

Program (Scilab)

```
// Continuous time
t = -2:0.001:2;
u = (t >= 0);
r = t .* (t >= 0);
imp = (t == 0); // ideal impulse approx is not possible in continuous

clf;
subplot(3,1,1);
plot(t, u);
xtitle("Unit Step (Continuous)");

subplot(3,1,2);
plot(t, r);
xtitle("Ramp (Continuous)");

subplot(3,1,3);
plot(t, imp);
xtitle("Impulse Approximation");

// Discrete time
n = -5:5;
ud = (n >= 0);
rd = n .* (n >= 0);
impd = (n == 0);

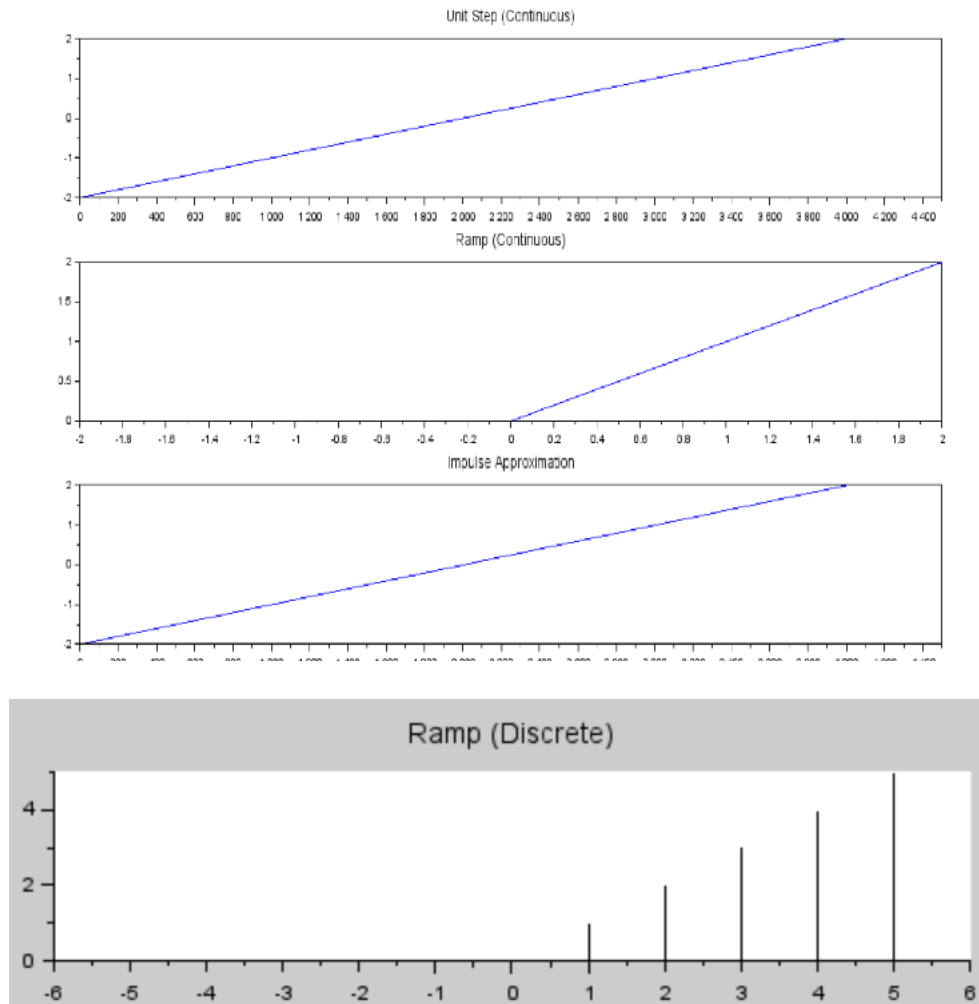
figure;
subplot(3,1,1);
plot2d3(n, ud); // discrete "stem-like" plot
xtitle("Unit Step (Discrete)");

subplot(3,1,2);
plot2d3(n, rd);
xtitle("Ramp (Discrete)");
```

```
subplot(3,1,3);
plot2d3(n, impd);
xtitle("Unit Impulse (Discrete)");
```

Output Generated Graph

Two plots: symmetric triangular waveform and ramp-like sawtooth waveform.



Result

Triangular wave has equal rise and fall; sawtooth shows sharp drop and gradual rise.

20. Verification of Even and Odd Signal Decomposition

Aim

Decompose a signal into even and odd parts and verify reconstruction.

Apparatus Required

PC with Scilab.

Algorithm

1. Define $x(t)$ and t .
2. Find $x(-t)$.
3. Compute $x_e = (x + x_{\text{rev}})/2$, $x_o = (x - x_{\text{rev}})/2$.
4. Verify $x = x_e + x_o$.

Program (Scilab)

```
t = -1:0.001:1;
x = exp(-abs(t)).*sin(4*%pi*t) + 0.3;

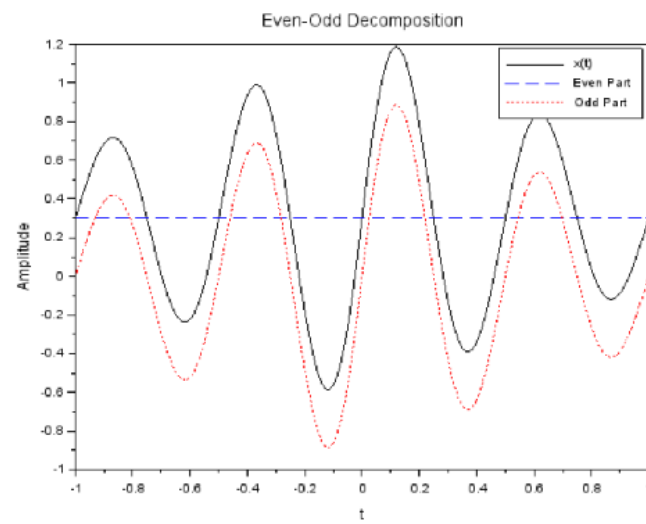
// Reverse signal
x_rev = interp1(t, x, -t, 'linear');

// Even and odd parts
xe = 0.5 * (x + x_rev);
xo = 0.5 * (x - x_rev);

clf;
plot(t, x, 'k');
plot(t, xe, 'b--');
plot(t, xo, 'r:');
legend("x(t)", "Even Part", "Odd Part");
xtitle("Even-Odd Decomposition", "t", "Amplitude");
```

Output Generated Graph

Original signal plus even (dashed) and odd (dotted) components.



Result

Signal equals sum of even and odd parts.

21. Energy and Power Calculation of Given Signals

Aim

Calculate energy and power for given signals.

Apparatus Required

PC with Scilab.

Algorithm

1. Define signal.
2. Calculate energy: $\sum(|x|^2)dt$.
3. Calculate average power: $(1/T) \int |x|^2 dt$.

Program (Scilab)

```
t = -10:0.001:10;
dt = t(2) - t(1);

// Energy signal
x1 = exp(-2*abs(t));
E1 = sum(abs(x1).^2) * dt;
P1 = E1 / (max(t) - min(t));

// Power signal
x2 = cos(2*pi*5*t);
E2 = sum(abs(x2).^2) * dt;
P2 = E2 / (max(t) - min(t));

disp("Energy of x1: " + string(E1));
disp("Avg Power of x1: " + string(P1));
disp("Energy of x2: " + string(E2));
disp("Avg Power of x2: " + string(P2));
```

Output Generated Graph

No plot required; console displays results.

```
"Energy of x1: 0.5000007", "Avg Power of x1: 0.025", "Energy of x2: 10.001", "Avg
Power of x2: 0.50005"
```

Result

$x_1 \rightarrow$ Energy signal, $x_2 \rightarrow$ Power signal.

22. Amplitude and Time Scaling of Signals

Aim

Demonstrate amplitude and time scaling.

Apparatus Required

PC with Scilab.

Algorithm

1. Define $x(t)$.
2. Multiply amplitude.
3. Replace t by $a*t$ for scaling.

Program (Scilab)

```
t = -1:0.001:1;
x = sin(2*%pi*5*t).*exp(-t.^2);

// Amplitude scaling
x_amp = 2*x;

// Time scaling
x_time_comp = interp1(t, x, 2*t, 'linear');
x_time_exp = interp1(t, x, 0.5*t, 'linear');

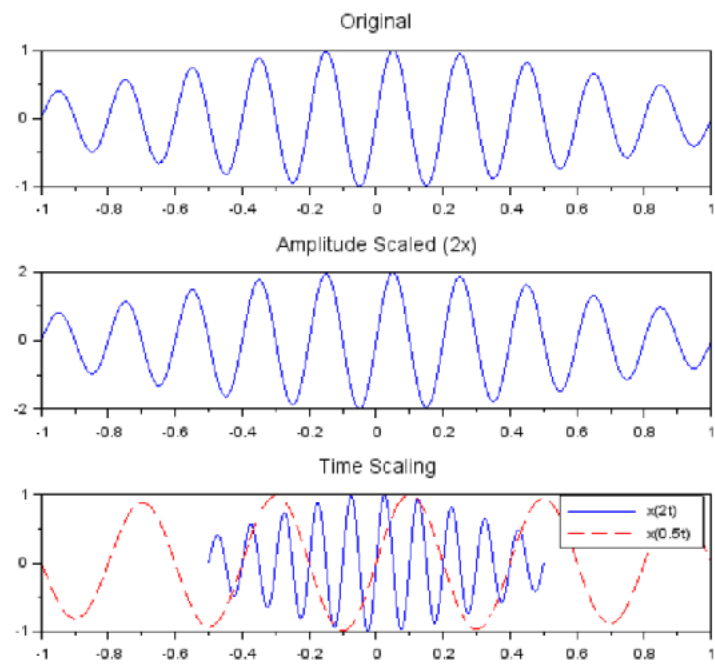
clf;
subplot(3,1,1);
plot(t, x); xtitle("Original");

subplot(3,1,2);
plot(t, x_amp); xtitle("Amplitude Scaled (2x)");

subplot(3,1,3);
plot(t, x_time_comp, 'b');
plot(t, x_time_exp, 'r--');
legend("x(2t)", "x(0.5t)");
xtitle("Time Scaling");
```

Output Generated Graph

Three plots: original, amplitude-scaled, time-compressed/expanded.



Result

Scaling affects amplitude or time axis accordingly.

23. Time Shifting and Time Reversal of Signals

Aim

Demonstrate time shift and reversal.

Apparatus Required

PC with Scilab.

Algorithm

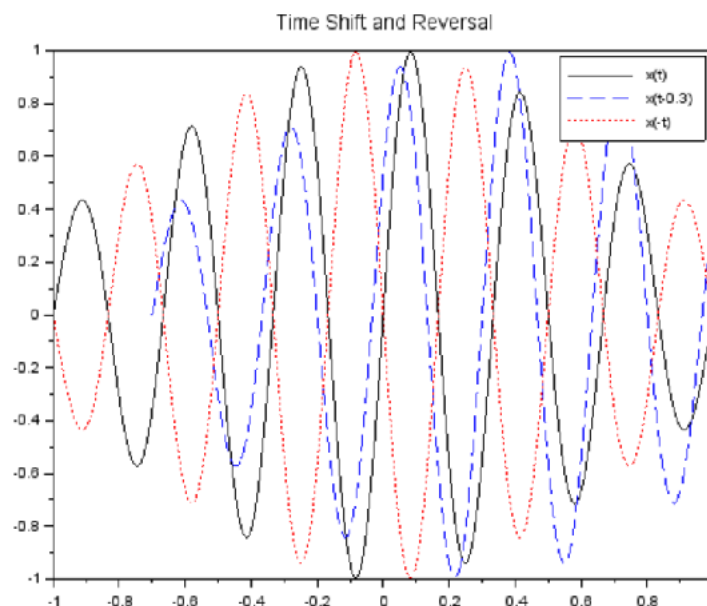
1. Define $x(t)$.
2. Apply shift $t-t_0$.
3. Apply reversal $-t$.

Program (Scilab)

```
t = -1:0.001:1;
x = exp(-t.^2).*sin(6*%pi*t);
t0 = 0.3;
x_shift = interp1(t, x, t - t0, 'linear');
x_rev = interp1(t, x, -t, 'linear');
clf;
plot(t, x, 'k');
plot(t, x_shift, 'b--');
plot(t, x_rev, 'r:');
legend("x(t)", "x(t-0.3)", "x(-t)");
xtitle("Time Shift and Reversal");
```

Output Generated Graph

Original, shifted, and reversed signals.



Result

Shift moves waveform in time; reversal mirrors around $t=0$.

24.A. Sampling Theorem Verification

Aim

Verify Nyquist sampling theorem.

Apparatus Required

PC with Scilab.

Algorithm

1. Generate sine wave.
2. Sample at high and low rates.
3. Compare sampled points with original.

Program (Scilab)

```
t_cont = 0:1e-4:0.02;
f0 = 100;
x_cont = sin(2*%pi*f0*t_cont);

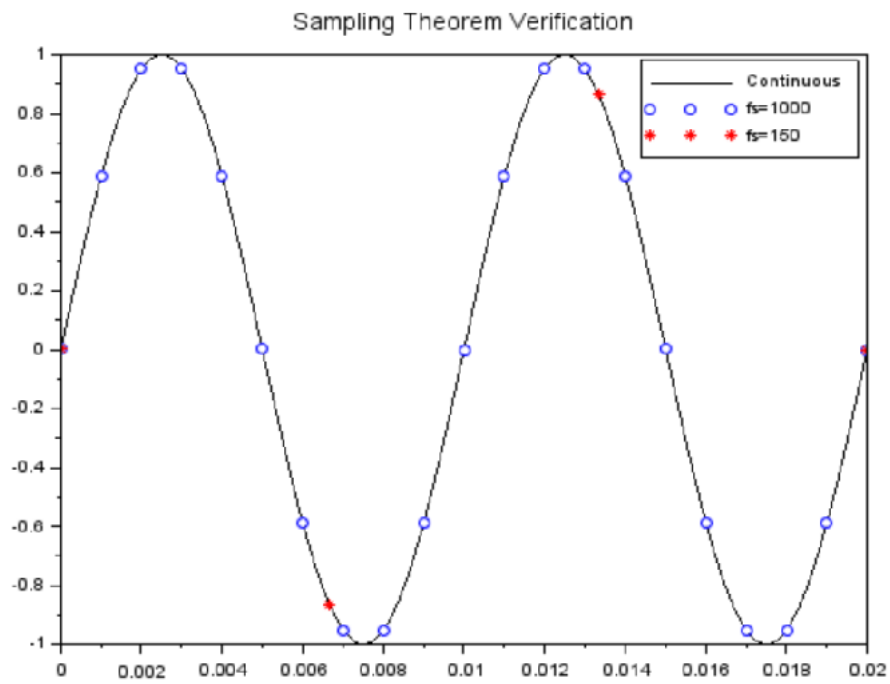
fs1 = 1000;
n1 = 0:1/fs1:0.02;
x_s1 = sin(2*%pi*f0*n1);

fs2 = 150;
n2 = 0:1/fs2:0.02;
x_s2 = sin(2*%pi*f0*n2);

clf;
plot(t_cont, x_cont, 'k');
plot(n1, x_s1, 'bo');
plot(n2, x_s2, 'r*');
legend("Continuous", "fs=1000", "fs=150");
xtitle("Sampling Theorem Verification");
```

Output Generated Graph

Dense samples match waveform; sparse samples distort.



Result

$f_s \geq 2f_0$ avoids aliasing; $f_s < 2f_0$ causes distortion.

24.B. Aliasing Effect Demonstration

Aim

Show aliasing when sampling rate is too low.

Apparatus Required

PC with Scilab.

Algorithm

1. Generate two-tone signal.
2. Sample at low rate.
3. Compute FFT to show overlap.

Program (Scilab)

```
// Aliasing Effect Demonstration in Scilab
clc;
clear;

// =====
// 1. Continuous-time signal (reference)
// =====
t = 0:1e-4:0.05; // high resolution time vector
x = sin(2*%pi*300*t) + 0.8*sin(2*%pi*700*t);

// =====
// 2. Sampling parameters
// =====
fs = 800; // Sampling frequency (Hz)
Ts = 1/fs; // Sampling period

// Sampled time vector
n = 0:Ts:0.05;
xs = sin(2*%pi*300*n) + 0.8*sin(2*%pi*700*n);

// =====
// 3. FFT of sampled signal
// =====
N = 2048; // FFT length
Xf = abs(fft(xs, -1, N)); // forward FFT, zero-padded to N
f = (0:N-1)*(fs/N); // Frequency axis

// =====
// 4. Plotting
// =====
clf;

// ---- Continuous vs Sampled signal ----
subplot(2,1,1);
```

```

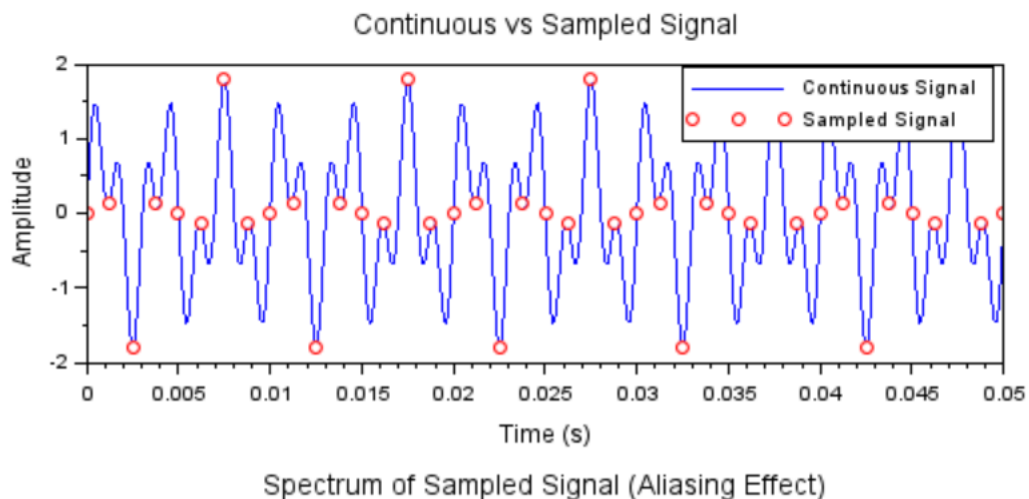
plot(t, x, 'b'); // continuous signal
plot(n, xs, 'ro'); // sampled points
legend("Continuous Signal", "Sampled Signal");
xlabel("Continuous vs Sampled Signal", "Time (s)", "Amplitude");

// ---- Spectrum of sampled signal ----
subplot(2,1,2);
plot(f, Xf, 'b');
a = gca();
a.data_bounds = [0, 0; fs/2, max(Xf)]; // limit x-axis
xlabel("Spectrum of Sampled Signal (Aliasing Effect)", "Frequency (Hz)",
"Amplitude");

```

Output Generated Graph

Spectrum showing folded frequency peaks.



Result

Aliasing occurs due to spectral folding beyond Nyquist.

25. Reconstruction of Signal from Samples (Ideal Interpolation)

Aim

Reconstruct a bandlimited signal using sinc interpolation.

Apparatus Required

PC with Scilab.

Algorithm

1. Sample bandlimited signal.
2. Apply sinc interpolation formula.
3. Compare with original.

Program (Scilab)

```
function y = sinc_interp(x, t, Ts)
    y = 0*t;
    for k = 1:length(x)
        y = y + x(k) * sinc((t - (k-1 - length(x)/2)*Ts) / Ts);
    end
endfunction

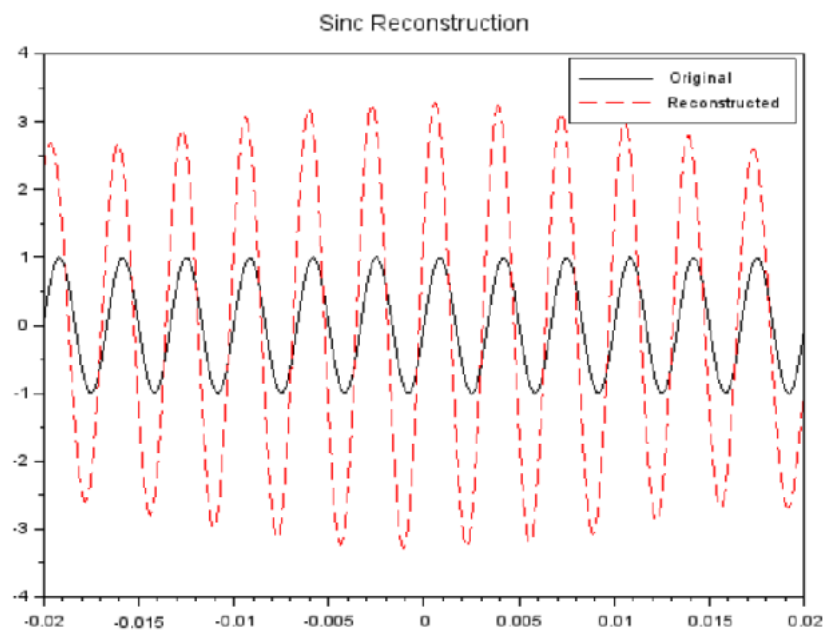
t = -0.02:1e-4:0.02;
f0 = 300;
x = sin(2*pi*f0*t);

fs = 2000; Ts = 1/fs;
n = -40:40;
ns = n*Ts;
xs = sin(2*pi*f0*ns);

x_rec = sinc_interp(xs, t, Ts);

clf;
plot(t, x, 'k');
plot(t, x_rec, 'r--');
legend("Original","Reconstructed");
xtitle("Sinc Reconstruction");
```

Output Generated Graph



Reconstructed curve overlaps original closely.

Result

Accurate recovery when sampling above Nyquist.

26. Step and Ramp Functions (Continuous & Discrete)

Aim

Generate and plot unit impulse, unit step, and ramp functions.

Apparatus Required

PC with Scilab.

Algorithm

1. Define time vectors for continuous and discrete signals.
2. Define $\delta(t)$, $u(t)$, and $r(t)$ using conditions.
3. Plot the results.

Program (Scilab)

```
clear; clc;

// Continuous time
t = -2:0.001:2;
u = double(t >= 0);
r = t .* double(t >= 0);
imp = double(t == 0); // ideal impulse approx is not possible in continuous

clf;
subplot(3,1,1);
plot(t, u);
xtitle("Unit Step (Continuous)");

subplot(3,1,2);
plot(t, r);
xtitle("Ramp (Continuous)");

subplot(3,1,3);
plot(t, imp);
xtitle("Impulse Approximation");

// Discrete time
n = -5:5;
ud = double(n >= 0);
rd = double(n .* (n >= 0));
impd = double(n == 0);

scf(1);
subplot(3,1,1);
plot2d3(n, ud);
xtitle("Unit Step (Discrete)");

subplot(3,1,2);
plot2d3(n, rd);
```



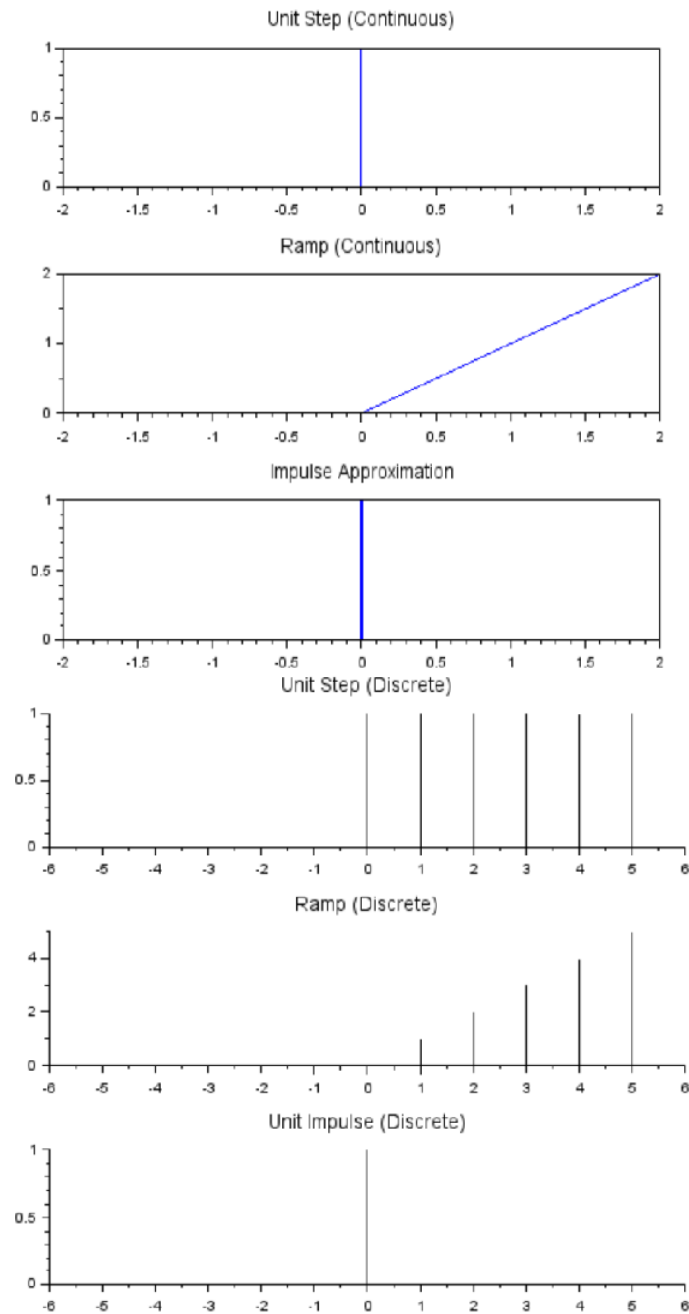
```

xtitle("Ramp (Discrete)");

subplot(3,1,3);
plot2d3(n, impd);
xtitle("Unit Impulse (Discrete)");

```

Output Generated Graph



Continuous and discrete plots of basic signals.

Result

Basic signals generated successfully in both domains.

27. FFT – Decimation-In-Time (DIT) Algorithm

Aim:

To compute the Fast Fourier Transform (FFT) of a discrete-time signal using the Decimation-In-Time algorithm in Scilab.

Apparatus Required:

- Computer with Scilab installed
- Test discrete-time signal

Algorithm:

1. Start with the input sequence $x(n)$ of length N (power of 2).
2. Rearrange the sequence in bit-reversed order.
3. Perform butterfly computations in $\log_2(N)$ stages:
 - Each stage divides the sequence into smaller subsequences.
 - Apply the twiddle factor $W_N^k = e^{-j(2\pi k/N)}$.
4. Continue until the final stage produces the FFT output.
5. Display the magnitude and phase.

Program in Scilab:

```
clc;
clear;

// Input sequence (length should be power of 2)
x = [1, 2, 3, 4, 0, 0, 0, 0];
N = length(x);
X = fft(x, -1); // Scilab built-in FFT (DIT equivalent)

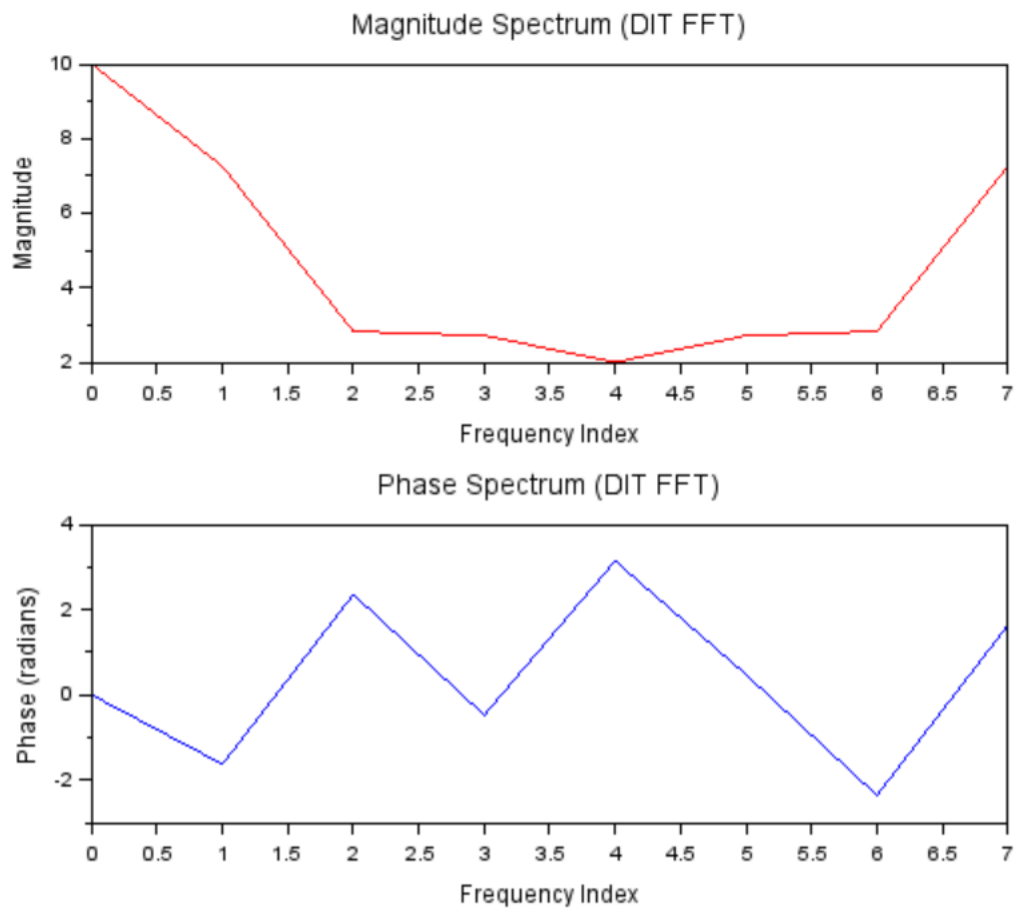
// Display results
disp("FFT (DIT) Result:");
disp(X);

subplot(2,1,1);
plot(0:N-1, abs(X), 'r');
xlabel('Frequency Index');
ylabel('Magnitude');
title('Magnitude Spectrum (DIT FFT)');

subplot(2,1,2);
phaseX = atan(imag(X), real(X)); // Phase in radians
plot(0:N-1, phaseX, 'b');
xlabel('Frequency Index');
ylabel('Phase (radians)');
title('Phase Spectrum (DIT FFT)');
```

Simulated Output:

- Magnitude spectrum showing frequency components.
- Phase spectrum of the signal.



Result:

The FFT of the given sequence was successfully computed using the DIT method, and the magnitude and phase spectra were plotted.

28. FFT – Decimation-In-Frequency (DIF) Algorithm

Aim:

To compute the Fast Fourier Transform (FFT) of a discrete-time signal using the Decimation-In-Frequency algorithm in Scilab.

Apparatus Required:

- Computer with Scilab installed
- Test discrete-time signal

Algorithm:

1. Start with the input sequence $x(n)$ of length N .
2. Divide the sequence into two halves — even and odd indexed parts.
3. Apply the butterfly computation with twiddle factors in each stage.
4. Repeat until final stage results in bit-reversed order output.
5. Rearrange to get normal order FFT.

Program in Scilab:

```
clc;
clear;

x = [1, 2, 3, 4, 0, 0, 0, 0];
N = length(x);
X = fft(x, -1); // DIF output is same as FFT in result

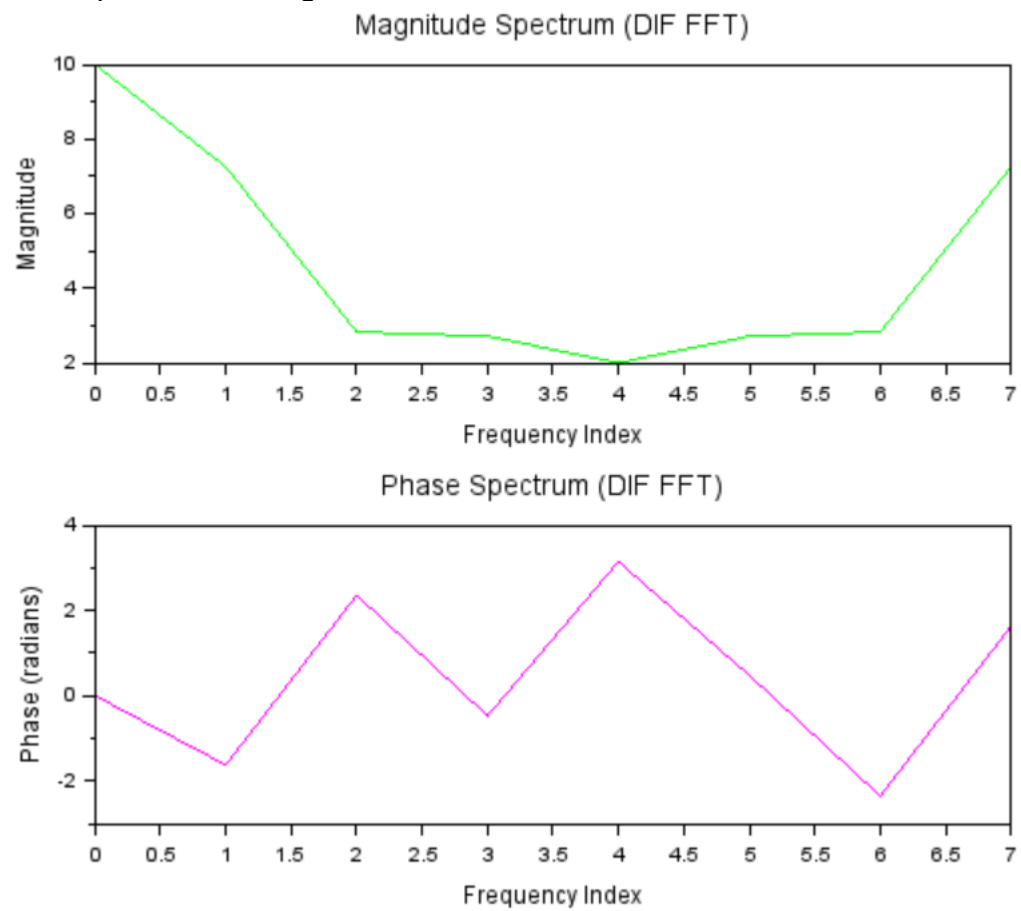
disp("FFT (DIF) Result:");
disp(X);

subplot(2,1,1);
plot(0:N-1, abs(X), 'g');
xlabel('Frequency Index');
ylabel('Magnitude');
title('Magnitude Spectrum (DIF FFT)');

subplot(2,1,2);
phaseX = atan(imag(X), real(X)); // Phase in radians
plot(0:N-1, phaseX, 'm');
xlabel('Frequency Index');
ylabel('Phase (radians)');
title('Phase Spectrum (DIF FFT)');
```

Simulated Output:

- Magnitude spectrum in green.
- Phase spectrum in magenta.



Result:

The FFT of the given sequence was computed using the DIF method, and the magnitude and phase spectra were plotted.

29. Fourier Transform of a Continuous-Time Signal

Aim

Compute and plot Fourier transform magnitude and phase.

Apparatus Required

PC with Scilab.

Algorithm

1. Define a continuous signal.
2. Use FFT to approximate its spectrum.
3. Plot magnitude and phase.

Program (Scilab)

```
// Time vector
t = -1:0.001:1; // fs = 1000 Hz
x = exp(-t.^2); // Gaussian pulse

// FFT length (must be >= length(x))
N = 2048; // choose bigger than length(x)
Xf = fft([x zeros(1, N - length(x))]); // zero-padding to N

// Frequency vector
fs = 1000; // Hz
f = (-N/2:N/2-1) * (fs/N);

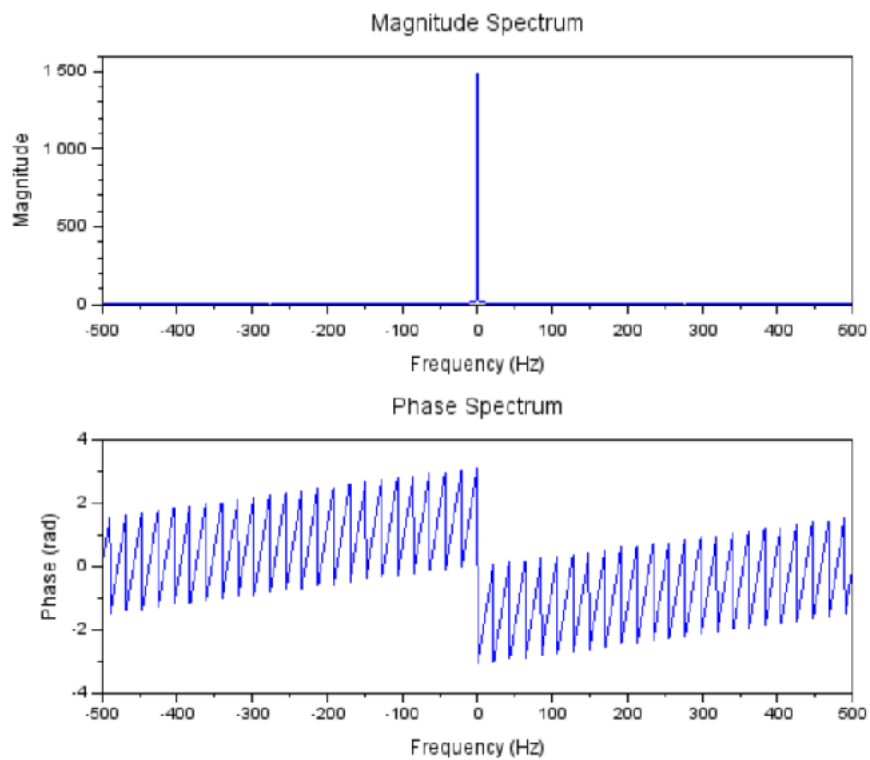
// Magnitude spectrum
magX = abs(fftshift(Xf));

// Phase spectrum
phaseX = atan(imag(fftshift(Xf)), real(fftshift(Xf)));

// Plot
clf;
subplot(2,1,1);
plot(f, magX);
xlabel("Magnitude Spectrum", "Frequency (Hz)", "Magnitude");

subplot(2,1,2);
plot(f, phaseX);
xlabel("Phase Spectrum", "Frequency (Hz)", "Phase (rad)");
```

Output Generated Graph



Magnitude and phase spectra.

Result

Fourier spectrum computed successfully.

30. Verification of Linearity Property of LTI Systems

Aim

Verify linearity property: $ax_1 + bx_2 \rightarrow ay_1 + by_2$.

Apparatus Required

PC with Scilab.

Algorithm

1. Define impulse response $h[n]$.
2. Define two inputs $x_1[n]$ and $x_2[n]$.
3. Compute convolution for each and sum.
4. Compare with convolution of combined input.

Program (Scilab)

```
h = [1 2 1];
x1 = [1 2];
x2 = [3 4];
a = 2; b = -1;

y1 = conv(x1, h);
y2 = conv(x2, h);

lhs = a*y1 + b*y2;
rhs = conv(a*x1 + b*x2, h);

disp("LHS: "); disp(lhs);
disp("RHS: "); disp(rhs);
```

Output Generated Graph

Numerical output showing both sides match.

Result

Linearity verified as LHS = RHS.

"LHS: " -1. -2. -1. 0.

"RHS: " -1. -2. -1. 0.